

UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y
TELECOMUNICACIONES

NOTA PARA COMPLETAR: Insertar acá el escudo de la UAGRM / FICCT, como en el formato de la materia.

Primer Parcial

**“HERRAMIENTA CASE COLABORATIVA CON IA LOCAL Y EN LA NUBE PARA EL
DISEÑO DE DATOS Y LA GENERACIÓN DE BACKEND SPRING BOOT A PARTIR DE
DIAGRAMAS UML”**

GitHub – Proyecto	Software Web	Guía de Usuario y Video
(insertar QR)	(insertar QR)	(insertar QR)

NOTA PARA COMPLETAR: Generar los tres códigos QR y pegarlos en la tabla de arriba: el repositorio de GitHub del proyecto, la URL donde quede publicado el software y el enlace al video de la guía de usuario. Debajo de cada QR conviene poner también el enlace en texto, por si el QR no se puede escanear.

ESTUDIANTE: RODRÍGUEZ, EDUARDO

REGISTRO: (completar con tu número de registro)

MATERIA: INGENIERÍA DE SOFTWARE I

GRUPO: (completar con tu grupo)

DOCENTE: ING. MARTÍNEZ CANEDO ROLANDO ANTONIO

SEMESTRE: 2 – 2026

SANTA CRUZ – BOLIVIA

ÍNDICE

Índice: en Word, clic derecho sobre esta tabla y "Actualizar campo" (o F9) para generarlo.

1. PERFIL DEL PROYECTO

1.1 INTRODUCCIÓN

El desarrollo de software avanza hacia entornos donde el modelado, la generación de código y la colaboración entre personas dejan de ser etapas separadas y pasan a ocurrir sobre un mismo artefacto vivo. Este proyecto se sitúa en ese terreno: es una herramienta de Ingeniería de Software Asistida por Computadora (CASE) que toma un diagrama de clases UML como única fuente de verdad y, a partir de él, cubre dos fases del proceso de desarrollo: el diseño de datos y la implementación del backend.

La herramienta se administra de forma colaborativa. Varias personas trabajan al mismo tiempo sobre el mismo diagrama, y el sistema está construido específicamente para resolver los problemas que esa simultaneidad produce: escrituras concurrentes sobre el mismo elemento, latencia, pérdida de conexión y divergencia entre lo que ve cada participante. La decisión de diseño central es que ningún cliente envía el documento completo: cada acción del usuario viaja como una operación granular a la que el servidor asigna un número de secuencia, y ese orden total es el que comparten todos.

La interfaz está pensada para ser conducida por inteligencia artificial además de por los controles tradicionales. En el escritorio se puede dictar o escribir una instrucción en lenguaje natural —“crea una clase Mascota con nombre texto y edad entero”— y el sistema la traduce en operaciones sobre el diagrama. En el teléfono la aplicación no tiene interfaz gráfica: es un asistente de voz con un solo control, que responde hablando. La capa de IA es intercambiable y funciona con un modelo local, con un modelo en la nube, o con el local y la nube como respaldo.

Sobre el diagrama, la herramienta recorre la fase de diseño de datos completa: construye el modelo conceptual, aplica las reglas de mapeo objeto-relacional de Rumbaugh, deriva el esquema lógico, analiza la normalización hasta la tercera forma normal justificando cada observación, y emite el DDL ejecutable. Después genera el backend en Spring Boot —entidades JPA, repositorios, servicios, controladores REST y el proyecto Maven completo— junto con una colección de Postman para probarlo.

El resultado es un entorno donde el paso del modelo al software funcionando no depende de transcribir a mano lo que ya está dibujado, y donde ese trabajo puede hacerse entre varias personas, hablando, y sin depender de tener conexión permanente.

1.2 OBJETIVO GENERAL

Desarrollar una herramienta CASE colaborativa que, a partir de un diagrama de clases UML 2.5 construido de forma visual, por voz, por texto o reconocido desde una imagen, cubra la fase de diseño de datos —modelado conceptual, mapeo objeto-relacional con las reglas de Rumbaugh, esquema lógico y normalización— y genere automáticamente el backend en Spring Boot conectado a una base de datos relacional; administrable de manera colaborativa con resolución de conflictos y funcionamiento ante desconexión, y operable mediante

inteligencia artificial local y en la nube tanto desde el escritorio como desde un asistente de voz en el teléfono.

1.3 OBJETIVOS ESPECÍFICOS

- Implementar un editor de diagramas de clases UML 2.5 que soporte atributos con tipo y visibilidad, los cinco tipos de relación y las multiplicidades, incluyendo la resolución de relaciones muchos a muchos mediante clase asociativa.
- Construir un backend propio que persista los diagramas y sostenga la colaboración, eliminando la dependencia de servicios de terceros y permitiendo el funcionamiento en red local.
- Resolver los problemas propios del trabajo concurrente mediante operaciones granulares con orden total, aplicación optimista, idempotencia por identificador de operación y marcas de borrado, evitando que dos participantes se sobrescriban.
- Garantizar el funcionamiento ante desconexión persistiendo cada operación en el dispositivo antes de enviarla, y reconciliando la cola pendiente al reconectar.
- Integrar una capa de inteligencia artificial intercambiable entre un modelo local y uno en la nube, que interprete instrucciones en lenguaje natural y las convierta en operaciones sobre el diagrama.
- Incorporar el reconocimiento de diagramas a partir de fotografías, bocetos o capturas, transformándolos en modelos UML editables.
- Desarrollar una aplicación móvil sin interfaz gráfica, operada íntegramente por voz, que trabaje sobre la misma pizarra compartida.
- Implementar el módulo de diseño de datos que derive del diagrama el modelo conceptual, el mapeo relacional según las reglas de Rumbaugh, el análisis de normalización hasta 3FN y el DDL ejecutable.
- Generar el backend en Spring Boot con entidades, repositorios, servicios y controladores REST, junto con la colección de pruebas para Postman.
- Habilitar el intercambio con otras herramientas CASE mediante un formato estándar de UML, dejando la arquitectura preparada para incorporar formatos propietarios.
- Documentar el proceso de desarrollo bajo el Proceso Unificado de Desarrollo de Software (PUDS), registrando fases, flujos de trabajo e iteraciones.

1.4 DESCRIPCIÓN DEL PROBLEMA

El diseño de un sistema empieza casi siempre en un diagrama, y termina casi siempre en código escrito a mano que repite lo que el diagrama ya decía. Esa traducción manual es lenta, mecánica y es donde se introducen inconsistencias: el código se modifica, el diagrama no, y a partir de ese momento la documentación describe un sistema que ya no existe. Una herramienta CASE ataca ese problema tratando al modelo como la fuente de verdad desde la cual se deriva el resto.

El problema se vuelve más difícil cuando el modelado es una actividad de equipo. Si dos personas editan el mismo diagrama al mismo tiempo y cada cliente guarda el documento completo, el último en escribir borra el trabajo del otro sin que nadie se entere. Esa pérdida

silenciosa es el riesgo principal de una pizarra compartida, y no se resuelve con avisos en la interfaz: hay que cambiar la unidad de sincronización.

A eso se suma la conectividad. En un aula, una oficina o un teléfono en la calle, la red se corta. Si el trabajo de los últimos minutos vive solo en la memoria del navegador, se pierde al recargar; y si al reconectar se reenvía todo sin control, se duplica. Hace falta entonces que cada cambio sea durable localmente antes de viajar, y que reenviarlo sea inofensivo.

Un cuarto problema es la forma de interactuar. Modelar con el mouse obliga a tener las dos manos y la vista en la pantalla, lo que excluye situaciones tan comunes como discutir un modelo caminando o frente a una pizarra física. Una interfaz conducida por voz y por lenguaje natural cambia eso, pero introduce su propia dificultad: depender de un servicio en la nube significa que sin internet —o sin saldo— la función no existe, lo que es inaceptable para una herramienta que debe funcionar de manera local.

Finalmente, el diseño de datos suele quedar como un paso implícito. Del diagrama de clases al esquema relacional hay un conjunto de reglas de transformación bien establecidas, y decisiones de normalización que conviene justificar antes de crear las tablas. Cuando ese paso se hace de memoria, aparecen columnas redundantes y dependencias transitivas que después cuestan caro.

Este proyecto aborda esos cinco problemas como un solo sistema: el modelo como fuente de verdad, la colaboración sin pérdida de trabajo, la durabilidad ante desconexión, la operación por voz con inteligencia artificial local, y el diseño de datos explícito y justificado.

1.5 ALCANCE

El sistema se organiza en seis módulos. Todos son accesibles por cualquier participante de la pizarra; el sistema no implementa por el momento autenticación ni permisos diferenciados, lo que se declara explícitamente en el apartado de limitaciones conocidas.

Módulo de gestión de pizarras y sesión colaborativa

Permite crear, renombrar y eliminar pizarras, compartir el enlace de invitación y trabajar simultáneamente sobre el mismo diagrama. Incluye presencia de participantes —nombre, color e indicación de qué clase está tocando cada uno—, orden total de las operaciones, reconciliación al reconectar y bitácora de auditoría de quién hizo qué.

Módulo de modelado UML

Editor del diagrama de clases con atributos tipados y su visibilidad, los cinco tipos de relación de UML 2.5 —asociación, agregación, composición, herencia y dependencia—, multiplicidades, y generación asistida de la clase asociativa cuando se declara una relación muchos a muchos.

Módulo de interfaz asistida por inteligencia artificial

Traduce instrucciones en lenguaje natural, escritas o dictadas, en operaciones sobre el diagrama. Reconoce diagramas a partir de fotografías, bocetos y capturas de pantalla. La capa de IA admite un proveedor local, uno en la nube, o el local con el de la nube como respaldo automático. Incluye además la guía de usuario en forma de agente, que responde preguntas sobre la herramienta.

Módulo móvil por voz

Aplicación web instalable, sin interfaz gráfica: un único control de micrófono y respuesta hablada. Opera sobre la misma pizarra compartida que el escritorio y comparte con él el mecanismo de cola offline.

Módulo de diseño de datos

Deriva del diagrama de clases el modelo conceptual con sus cardinalidades, aplica las reglas de mapeo objeto-relacional, produce el esquema lógico con claves primarias y foráneas, analiza el cumplimiento de la primera, segunda y tercera forma normal con la justificación de cada observación, y emite el DDL para PostgreSQL y MySQL. Todo este módulo es determinístico: no interviene la inteligencia artificial, de modo que el resultado es reproducible y defendible paso por paso.

Módulo de generación de software

Genera el proyecto Maven completo del backend en Spring Boot —entidades JPA con sus claves foráneas y reglas de borrado, repositorios, servicios, controladores REST, configuración y clase principal—, la colección de Postman para probar los endpoints y un frontend de prueba. El proyecto generado se entrega empaquetado y listo para ejecutar.

Módulo de interoperabilidad con otras herramientas CASE

Exportación e importación en XMI 2.5, el formato de intercambio estándar de UML definido por el OMG, y en el formato JSON propio de la herramienta. La arquitectura de adaptadores está preparada para incorporar formatos propietarios; el adaptador nativo de Architech queda pendiente y su estado se detalla en las limitaciones conocidas.

Parte I – Fundamentación Teórica

Esta parte explica qué se usó en el desarrollo, cómo se usó y por qué se eligió. El orden sigue los puntos obligatorios del enunciado, y cada apartado cierra con la aplicación concreta en este proyecto, de modo que la teoría quede vinculada a decisiones verificables en el código.

1. Ingeniería de Software Asistida por Computadora (CASE)

La Ingeniería de Software Asistida por Computadora es el conjunto de métodos y herramientas informáticas que dan soporte a las actividades del ciclo de vida del software. Su propósito no es reemplazar el juicio del ingeniero, sino trasladar a la máquina el trabajo mecánico y propenso a error: mantener la consistencia entre modelos, derivar artefactos unos de otros y centralizar la información del proyecto.

La idea que sostiene a una herramienta CASE es que el modelo es la fuente de verdad. Si el diagrama de clases describe las entidades del sistema, entonces el esquema de base de datos y las clases del backend no son documentos independientes que haya que mantener en paralelo, sino consecuencias del modelo. Cuando esa derivación se automatiza, la inconsistencia entre diseño e implementación deja de ser posible por construcción.

Clasificación de las herramientas CASE

- **Herramientas de análisis:** dan soporte a la captura de requisitos, la elaboración de casos de uso y el estudio de viabilidad.
- **Herramientas de modelado y diseño:** permiten construir y validar modelos visuales — típicamente UML— verificando su coherencia estructural.
- **Herramientas de generación de código:** transforman los modelos en código fuente ejecutable, automatizando capas completas de la implementación.
- **Herramientas de prueba y aseguramiento de la calidad:** automatizan la ejecución de pruebas y la verificación del cumplimiento de estándares.
- **Herramientas de mantenimiento e ingeniería inversa:** permiten reconstruir modelos a partir de sistemas existentes y sostener su evolución.

Una distinción habitual separa las herramientas por su cobertura: las de nivel superior (upper CASE) atienden las fases tempranas de análisis y diseño; las de nivel inferior (lower CASE) atienden la implementación y las pruebas; y las integradas (I-CASE) cubren el recorrido completo manteniendo un repositorio común.

Aplicación en este proyecto

La herramienta desarrollada es una CASE integrada de alcance acotado: cubre el diseño de datos y la implementación del backend a partir de un único modelo. Concretamente, se ubica en las cinco categorías de la siguiente manera:

- **Modelado y diseño:** el editor de diagramas de clases, con validación de multiplicidades y generación de clases asociativas.

- **Análisis:** el módulo de diseño de datos, que analiza el modelo y reporta observaciones de normalización con su justificación.
- **Generación de código:** el generador de backend Spring Boot y el generador del DDL relacional.
- **Prueba:** la generación de la colección de Postman a partir de los endpoints derivados del modelo.
- **Mantenimiento e intercambio:** la importación y exportación en XMI 2.5, que permite que el modelo entre y salga de otras herramientas sin transcribirlo.

El repositorio común que exige el concepto de CASE integrada está implementado como una base de datos relacional propia, donde el diagrama vive como documento versionado por un número de secuencia, acompañado de la bitácora completa de operaciones que lo produjeron.

2. Desarrollo de aplicaciones basado en componentes

El desarrollo basado en componentes (CBSD, por Component-Based Software Development) construye sistemas ensamblando unidades de software independientes, reemplazables y con una interfaz bien definida, en lugar de escribir cada sistema como un bloque monolítico. Un componente se caracteriza por lo que ofrece y lo que requiere, no por cómo está implementado por dentro.

Propiedades que definen un componente

- **Encapsulamiento:** expone una interfaz y oculta su implementación, de modo que se puede cambiar por dentro sin afectar a quien lo usa.
- **Reutilización:** resuelve un problema general y puede emplearse en más de un contexto.
- **Sustituibilidad:** puede reemplazarse por otra implementación que respete la misma interfaz.
- **Composición:** se combina con otros componentes para construir funcionalidad mayor.
- **Independencia de despliegue:** puede evolucionar y versionarse por separado.

Aplicación en este proyecto

El sistema está compuesto por componentes con responsabilidad única y frontera explícita. Los más relevantes, con su interfaz y su razón de ser:

Componente	Interfaz que expone	Por qué es un componente
Motor de colaboración (collab / rooms / ws)	Aplicar una operación sobre el documento y devolver si fue aceptada.	La lógica de resolución de conflictos es independiente de UML: opera sobre nodos y aristas genéricos y se puede probar sin interfaz.
Cola offline (offlineQueue)	Encolar, consultar y desencolar operaciones pendientes.	Aísla la durabilidad local del resto de la aplicación; se podría cambiar IndexedDB por otro almacenamiento sin tocar nada más.
Proveedor de IA	Recibir un prompt y devolver	Es el caso más claro de

(ai/provider)	texto, eligiendo proveedor.	sustituibilidad: el modelo local y el de la nube son dos implementaciones de la misma interfaz, intercambiables por configuración.
Adaptadores CASE (architech)	Exportar e importar un diagrama en un formato dado.	Cada formato es una implementación de la misma interfaz; agregar uno nuevo no obliga a modificar el editor.
Módulo de diseño de datos (dataDesign)	Dado un diagrama, devolver modelo conceptual, esquema, normalización y DDL.	Es una función pura sin estado ni efectos: se puede ejecutar y verificar de forma aislada, lo que permitió probar el DDL generado contra una base real.
Generadores de código (backend / frontend / postman)	Dado un diagrama, producir un paquete descargable.	Tres componentes distintos que consumen el mismo modelo de entrada.

El backend que el sistema genera también respeta este paradigma: se estructura en las capas de entidad, repositorio, servicio y controlador, donde cada capa depende únicamente de la interfaz de la siguiente. Esa separación es la que permite que el código generado sea comprensible y modificable por una persona, y no un bloque inmanejable.

Mantenimiento del software

Sobre esta base de componentes se aplican los cuatro tipos clásicos de mantenimiento:

- **Correctivo:** corrección de defectos detectados. Durante el desarrollo se corrigieron, entre otros, un DDL generado que no era ejecutable por un comentario mal ubicado y un indicador de conexión que quedaba desactualizado tras recuperar la red.
- **Adaptativo:** ajuste ante cambios del entorno. El caso principal fue la sustitución del servicio de backend externo por un backend propio sobre PostgreSQL.
- **Preventivo:** reducción de deuda técnica. Se eliminaron las credenciales incrustadas en el código fuente y se centralizaron las conversiones de modelo que estaban duplicadas.
- **Perfectivo:** incorporación de mejoras. La capa de IA local, el módulo de diseño de datos, el asistente de voz móvil y el sistema de temas son mantenimiento perfectivo sobre la base existente.

3. Arquitectura de software

La arquitectura de software es el conjunto de decisiones estructurales de un sistema: qué componentes lo forman, cómo se relacionan, qué reglas gobiernan esas relaciones y qué atributos de calidad se privilegian. Son decisiones costosas de revertir, y por eso se toman y se documentan de manera explícita.

Una arquitectura no se juzga por su elegancia sino por los atributos de calidad que consigue. En este sistema los atributos priorizados fueron, en este orden: consistencia de los datos

ante edición concurrente, disponibilidad sin conexión, y capacidad de sustituir el proveedor de inteligencia artificial.

Estilos arquitectónicos empleados

- **Cliente-servidor en capas:** el cliente concentra la presentación y el estado de la vista; el servidor concentra la autoridad sobre el documento y la persistencia. La capa de acceso a datos está aislada del transporte.
- **Arquitectura dirigida por eventos:** el canal WebSocket transporta operaciones, no estados. El servidor publica las operaciones aceptadas y cada cliente las aplica; el documento es el resultado de reproducir la secuencia.
- **Registro de operaciones (event log):** la tabla de operaciones conserva la historia completa y ordenada de los cambios, y el documento materializado es una proyección de ese registro. Esto da trazabilidad, permite poner al día a un cliente atrasado sin transferir todo, y constituye la bitácora de auditoría.
- **Patrón de estrategia en la capa de IA:** el proveedor concreto —local o en la nube— se resuelve en tiempo de ejecución según la configuración, detrás de una interfaz única.
- **Arquitectura offline-first en el cliente:** la escritura local es el camino primario y la sincronización es una consecuencia, no un requisito para operar.

Decisiones arquitectónicas y su justificación

Decisión	Alternativa descartada	Por qué
Sincronizar operaciones granulares con orden total asignado por el servidor.	Guardar el documento completo desde cada cliente.	Con documento completo, la última escritura sobrevive y borra el trabajo ajeno. Es el defecto que este diseño elimina por construcción.
Backend propio sobre PostgreSQL.	Servicio de base de datos en la nube de terceros.	El enunciado exige funcionamiento local y previsión de la desconexión; además elimina la dependencia de una cuenta externa y permite operar en red local.
Persistir cada operación en el cliente antes de enviarla.	Mantener los cambios pendientes solo en memoria.	En memoria, un corte de red seguido de una recarga pierde el trabajo. En IndexedDB sobrevive a la recarga y al cierre de la pestaña.
Identificador de operación generado por el cliente, con unicidad en la base.	Confirmación por contador secuencial del cliente.	Hace el reenvío idempotente: tras reconectar se puede reenviar la cola entera sin riesgo de duplicar.
Capa de IA en el servidor, no en el navegador.	Llamar al proveedor de IA desde el cliente.	Evita exponer credenciales en el navegador y permite que el escritorio y el móvil compartan el mismo prompt y el mismo proveedor.
Módulo de diseño de datos determinístico.	Delegar el mapeo y la normalización a la IA.	El resultado debe ser reproducible y justificable regla

		por regla; un modelo generativo no ofrece esa garantía.
--	--	---

[DIAGRAMA 01] Diagrama de arquitectura general del sistema

Qué debe mostrar: Las tres capas del sistema y los canales entre ellas: clientes (navegador de escritorio y PWA móvil), servidor (API REST, servidor WebSocket, capa de IA) y persistencia (PostgreSQL). Dibujar los dos canales por separado: HTTP/REST para pizarras, diagramas, IA e imágenes; y WebSocket para operaciones y presencia. Incluir los dos proveedores de IA como elementos externos: Ollama en la misma máquina y OpenAI en internet. Marcar IndexedDB dentro del cliente como el almacenamiento de la cola offline.

Cómo construirlo: Diagrama de componentes o de bloques. Un rectángulo por capa, con los componentes adentro; flechas etiquetadas con el protocolo. Señalar con línea discontinua la frontera de lo que sigue funcionando sin red (cliente + IndexedDB + Ollama).

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 1: Diagrama de arquitectura general del sistema".

4. UML 2.5, centrado en el diagrama de clases

UML (Unified Modeling Language) es el lenguaje de modelado estandarizado por el Object Management Group. La versión 2.5 reorganizó la especificación para hacerla más legible y definió catorce tipos de diagrama, divididos en estructurales —qué compone el sistema— y de comportamiento —qué hace el sistema.

El diagrama de clases es el diagrama estructural central y el que esta herramienta toma como entrada. Describe las clases del sistema, sus atributos y operaciones, y las relaciones entre ellas.

Elementos del diagrama de clases implementados

Una clase se representa como un rectángulo dividido en compartimentos: el nombre, los atributos y las operaciones. La herramienta implementa los dos primeros, que son los necesarios para derivar el esquema de datos y las entidades del backend.

Cada atributo lleva un tipo y una visibilidad. La visibilidad se anota con un símbolo: el signo más para público, el menos para privado y la almohadilla para protegido. Los tipos soportados son String, Integer, Float, Boolean y Date, que son los que tienen una correspondencia directa con tipos SQL y con tipos de Java.

Relaciones soportadas

Relación	Semántica UML	Notación	Consecuencia en el mapeo
Asociación	Vínculo estructural entre dos clases, sin dependencia de existencia.	Línea simple.	Clave foránea en el lado “muchos”, con restricción de integridad.

Agregación	Relación todo-parte donde la parte puede existir sin el todo.	Rombo hueco del lado del todo.	Clave foránea que admite nulos y se anula al borrar el todo (ON DELETE SET NULL).
Composición	Relación todo-parte con dependencia de existencia: la parte no vive sin el todo.	Rombo lleno del lado del todo.	Clave foránea obligatoria con borrado en cascada (ON DELETE CASCADE).
Generalización (herencia)	Una clase especializa a otra.	Triángulo hueco apuntando al padre.	Tabla por subclase con clave foránea única hacia la tabla padre.
Dependencia	Una clase usa a otra sin contenerla estructuralmente.	Línea discontinua con punta de flecha.	No genera estructura en el esquema relacional; documenta el acoplamiento.

Las multiplicidades indican cuántas instancias participan en cada extremo. La herramienta trabaja con los dos casos que determinan la estructura relacional: uno y muchos. Cualquier notación de muchos —n, m, 0..*, 1..*— se normaliza al asterisco, porque para decidir dónde va la clave foránea lo único que importa es si el extremo es singular o plural.

Un caso merece atención especial. UML permite declarar una asociación muchos a muchos directamente entre dos clases, pero el modelo relacional no puede representarla sin una tabla intermedia. Por eso la herramienta materializa esa relación como una clase asociativa: una clase marcada como tal, que referencia a las dos clases relacionadas y puede tener atributos propios —la fecha de una inscripción, la cantidad de un detalle de venta—. Esto no es una limitación de la herramienta sino el reconocimiento explícito de que la relación muchos a muchos con atributos es, conceptualmente, una entidad.

Aplicación en este proyecto

El diagrama de clases cumple en este sistema tres funciones simultáneas: es el artefacto que el usuario edita, es la entrada del módulo de diseño de datos y del generador de backend, y es la unidad de intercambio con otras herramientas CASE a través de XMI 2.5. Esa triple función es la razón por la cual se eligió el diagrama de clases y no otro como núcleo del sistema.

[DIAGRAMA 02] Diagrama de clases del propio sistema (modelo de dominio)

Qué debe mostrar: Las clases que forman el dominio de la herramienta: Board (pizarra), Diagram (diagrama), Operation (operación), Participant (participante), ClassNode (clase del diagrama), Attribute (atributo), RelationEdge (relación) y Upload (imagen archivada). Incluir atributos con tipo y visibilidad, y las relaciones con sus multiplicidades: Board 1—1 Diagram; Diagram 1—* Operation; Diagram 1—* ClassNode; ClassNode 1—* Attribute; Diagram 1—* RelationEdge; RelationEdge referencia dos ClassNode (origen y destino).

Cómo construirlo: Usar la propia herramienta del proyecto para dibujarlo y exportarlo: sirve además como demostración de la funcionalidad. Marcar la composición entre Diagram y Operation con rombo lleno, porque las operaciones no existen sin su diagrama.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 2: Diagrama de clases del propio sistema (modelo de dominio)".

5. El Proceso Unificado de Desarrollo de Software (PUDS)

El Proceso Unificado de Desarrollo de Software, formulado por Ivar Jacobson, Grady Booch y James Rumbaugh —“los tres amigos”—, es un marco de proceso iterativo e incremental para construir software. Se caracteriza por tres notas: está dirigido por casos de uso, está centrado en la arquitectura, y es iterativo e incremental.

Dirigido por casos de uso

Los casos de uso no son solo un documento de requisitos: son el hilo que atraviesa todo el proceso. Se capturan al principio, guían el análisis y el diseño, determinan qué se implementa en cada iteración y definen qué se prueba. Un caso de uso describe una interacción completa que produce valor observable para un actor.

Centrado en la arquitectura

La arquitectura se establece temprano y de forma explícita, porque es lo más costoso de cambiar después. En PUDS se construye una línea base arquitectónica durante la fase de elaboración, validada implementando los casos de uso de mayor riesgo.

Iterativo e incremental

El desarrollo avanza en iteraciones cortas, cada una de las cuales produce un incremento ejecutable. Cada iteración recorre todos los flujos de trabajo en distinta proporción: las primeras pesan más en requisitos y análisis, las últimas en implementación y prueba.

Las cuatro fases

Fase	Propósito	Hito de salida
Inicio	Delimitar el alcance, identificar los actores y los casos de uso principales, evaluar la viabilidad y los riesgos.	Objetivos del ciclo de vida: hay acuerdo sobre qué se va a construir.
Elaboración	Capturar la mayoría de los requisitos, establecer y validar la arquitectura, y mitigar los riesgos principales.	Arquitectura del ciclo de vida: la arquitectura está probada sobre los casos de uso críticos.
Construcción	Implementar el resto de los casos de uso e integrar incrementalmente hasta alcanzar la capacidad operativa.	Capacidad operativa inicial: el producto es utilizable.
Transición	Desplegar el producto, capacitar al usuario, corregir los defectos detectados en uso real.	Liberación del producto.

Los flujos de trabajo

Transversalmente a las fases, PUDS define cinco flujos de trabajo fundamentales: captura de requisitos, análisis, diseño, implementación y prueba. Todos están presentes en todas las fases; lo que cambia es su peso relativo.

La pregunta clave sobre la fase de elaboración

El enunciado plantea explícitamente esta cuestión: ¿en la fase de elaboración se pueden tener modelos pero no código? La respuesta es no, y entender por qué es central para comprender PUDS.

La fase de elaboración tiene como hito la validación de la arquitectura, y una arquitectura no se valida dibujándola: se valida ejecutándola. Por eso el producto de la elaboración es una línea base arquitectónica ejecutable, es decir, código que implementa los casos de uso de mayor riesgo técnico y demuestra que las decisiones estructurales funcionan. Lo que distingue a la elaboración de la construcción no es la ausencia de código, sino su propósito: en elaboración el código sirve para reducir riesgo y probar la arquitectura, mientras que en construcción sirve para completar funcionalidad sobre una arquitectura ya estable.

Sostener lo contrario equivaldría a aceptar el modelo en cascada con otro nombre: diseñar completo y recién después programar, que es precisamente lo que PUDS se propone evitar. En este proyecto la elaboración se cerró cuando el mecanismo de colaboración por operaciones con orden total quedó implementado y verificado con tres sesiones simultáneas, porque ese era el riesgo técnico principal; a partir de ahí, el resto de las funcionalidades fue construcción.

Aplicación en este proyecto

El desarrollo se organizó en tres iteraciones, cuyo detalle se documenta en la Parte II. La distribución de fases fue la siguiente:

Fase	Iteración	Contenido	Riesgo atacado
Inicio	Iteración 0	Análisis del sistema base heredado, identificación de actores y casos de uso, y evaluación de qué era reutilizable.	Sobreestimar lo aprovechable del código existente.
Elaboración	Iteración 1	Backend propio, motor de colaboración por operaciones con orden total, cola offline y presencia. Verificado con tres sesiones concurrentes.	Pérdida silenciosa de trabajo por escrituras concurrentes; pérdida de cambios ante desconexión.
Construcción	Iteración 2	Capa de IA local y en la nube, asistente por voz y texto, reconocimiento desde imagen, PWA móvil sin interfaz gráfica.	Dependencia de un servicio externo para una función que debe operar localmente.
Construcción	Iteración 3	Módulo de diseño de datos completo, interoperabilidad XMI 2.5, guía de usuario	Esquema relacional incorrecto o DDL no ejecutable.

		como agente, rediseño de la interfaz con tema claro y oscuro.	
Transición	Pendiente	Publicación de la documentación, defensa presencial y ajustes solicitados.	Inconsistencias entre la documentación y el software.

[DIAGRAMA 03] Ciclo de vida PUDS del proyecto

Qué debe mostrar: La matriz clásica de PUDS: en el eje horizontal las cuatro fases (inicio, elaboración, construcción, transición) subdivididas en las iteraciones del proyecto; en el eje vertical los cinco flujos de trabajo (requisitos, análisis, diseño, implementación, prueba). Las curvas de cada flujo deben mostrar que requisitos y análisis pesan más al principio, e implementación y prueba al final, pero que ninguno es cero en ninguna fase.

Cómo construirlo: Reproducir la figura estándar de PUDS con las iteraciones reales del proyecto rotuladas: Iteración 0, 1, 2 y 3 según la tabla anterior. Se puede dibujar en draw.io o en PowerPoint.

Herramienta: draw.io, Lucidchart o PowerPoint

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 3: Ciclo de vida PUDS del proyecto".

6. Técnica de Modelado de Objetos de Rumbaugh: reglas de mapeo

La Técnica de Modelado de Objetos (OMT, Object Modeling Technique) de James Rumbaugh aportó, entre otras cosas, un conjunto sistemático de reglas para transformar un modelo orientado a objetos en un esquema relacional. Esa transformación es necesaria porque los dos modelos no son equivalentes: el modelo de objetos expresa identidad, herencia y navegación por referencias, mientras que el modelo relacional expresa todo mediante valores en tablas y claves. La discrepancia entre ambos se conoce como desajuste objeto-relacional.

El valor de tener estas reglas escritas es que la transformación deja de ser una decisión de criterio individual y se vuelve un procedimiento reproducible y auditable. En este proyecto están implementadas como código determinístico, y la herramienta reporta, para cada elemento del esquema generado, qué regla lo produjo.

Reglas implementadas

Regla	Enunciado	Resultado en el esquema
R1	Toda clase concreta se transforma en una tabla, con una clave primaria subrogada.	CREATE TABLE con columna id de tipo serial como PRIMARY KEY.
R2	Cada atributo simple se transforma en una columna, con el tipo SQL equivalente a su tipo UML.	Columnas con tipo VARCHAR, INTEGER, DOUBLE PRECISION, BOOLEAN o DATE.
R3	La asociación uno a uno se implementa con una clave	Columna <clase>_id con UNIQUE.

	foránea en uno de los extremos, con restricción de unicidad.	
R4	La asociación uno a muchos se implementa con una clave foránea en el lado “muchos”.	Columna <clase>_id en la tabla del extremo plural.
R5	La asociación muchos a muchos se resuelve con una tabla intermedia cuya clave primaria es la unión de las dos foráneas.	Tabla <a>_ con PRIMARY KEY compuesta.
R6	La composición implica dependencia existencial: la foránea es obligatoria y el borrado se propaga.	NOT NULL y ON DELETE CASCADE.
R7	La agregación no implica dependencia existencial: la foránea admite nulos y se anula al borrar el agregado.	NULL permitido y ON DELETE SET NULL.
R8	La herencia se implementa con una tabla por subclase y una foránea única hacia la tabla padre.	Columna <padre>_id con UNIQUE y ON DELETE CASCADE.
R9	La clase asociativa se transforma en tabla con las foráneas a las dos clases que relaciona, conservando sus atributos propios.	Tabla con clave primaria propia y dos claves foráneas.

Sobre la estrategia de herencia elegida

Para la herencia existen tres estrategias habituales: una tabla por jerarquía con una columna discriminadora, una tabla por clase concreta con las columnas del padre repetidas, y una tabla por subclase con clave foránea al padre. Se eligió la tercera porque no introduce columnas nulas masivas —problema de la primera— ni duplica la definición de los atributos heredados —problema de la segunda—, y porque preserva la estructura de la jerarquía en el esquema, lo que la hace más fácil de leer contra el diagrama de origen. Su costo es la necesidad de una unión para recuperar una instancia completa, que se consideró aceptable.

Aplicación en este proyecto

El módulo de diseño de datos aplica estas nueve reglas y emite, junto con el esquema, una tabla de trazabilidad que indica para cada resultado cuál fue el elemento de origen y qué regla se aplicó. El DDL generado se verificó ejecutándolo contra una base PostgreSQL real, comprobando además el comportamiento semántico: que la composición efectivamente propague el borrado y que la agregación efectivamente anule la referencia.

7. Inteligencia artificial en el desarrollo de software

La inteligencia artificial se ha incorporado al ciclo de vida del software en dos roles distintos que conviene no confundir. En este apartado se trata el primero: la IA como asistente del

desarrollador, es decir, como herramienta que acelera la construcción del software. El apartado siguiente trata el segundo: la IA como parte de la funcionalidad del producto entregado.

Aplicaciones de la IA como asistente de desarrollo

- **Generación y completado de código:** a partir de una descripción en lenguaje natural o del contexto del archivo.
- **Refactorización asistida:** detección de duplicación, extracción de funciones, simplificación de estructuras.
- **Revisión de código:** detección de defectos, problemas de seguridad y desviaciones de los estándares del proyecto.
- **Generación de pruebas:** derivación de casos de prueba a partir del código o de la especificación.
- **Documentación:** generación y mantenimiento de documentación técnica derivada del código.
- **Traducción entre representaciones:** de un esquema a un modelo, de un modelo a código, de un formato a otro.

Limitaciones que condicionan su uso

Un modelo de lenguaje produce la continuación más probable, no la correcta. De ahí se derivan tres limitaciones que tuvieron consecuencias directas en el diseño de este sistema. Primero, la salida no es determinística: la misma entrada puede producir resultados distintos, lo que es inaceptable para una transformación que debe ser auditable. Segundo, el modelo puede producir resultados plausibles pero falsos, por lo que su salida debe validarse antes de aplicarse. Tercero, un modelo en la nube introduce una dependencia externa y un costo por uso.

Aplicación en este proyecto

Esas limitaciones determinaron una división de responsabilidades explícita, que es posiblemente la decisión de diseño más importante del sistema:

Tarea	¿Se usa IA?	Por qué
Interpretar una instrucción en lenguaje natural o voz y convertirla en operaciones sobre el diagrama.	Sí	Es exactamente el problema que la IA resuelve bien: lenguaje ambiguo a estructura. Y el resultado es verificable, porque el usuario ve el cambio en el diagrama al instante y puede deshacerlo.
Reconocer un diagrama en una fotografía o boceto.	Sí	Requiere visión por computadora; no hay alternativa determinística razonable. El resultado se presenta como punto de partida editable, no como verdad final.
Responder preguntas sobre el uso de la herramienta.	Sí, en segundo lugar	Primero se busca en una base de conocimiento escrita; la IA

		solo interviene si esa búsqueda no encuentra respuesta, y recibe esa misma base como contexto para no inventar funciones inexistentes.
Mapear el diagrama al esquema relacional.	No	Las reglas de Rumbaugh son un procedimiento cerrado. Debe producir el mismo resultado siempre y ser justificable regla por regla.
Analizar la normalización.	No	Cada observación debe venir con su justificación formal y ser discutible; una respuesta generativa no es auditable.
Generar el código del backend.	No	La correspondencia entre modelo y código es una plantilla determinística. Usar IA introduciría variabilidad sin ningún beneficio.

La regla general que se siguió es: la inteligencia artificial se usa para interpretar entradas ambiguas del usuario, y nunca para producir artefactos que deban ser reproducibles o auditables.

8. Inteligencia artificial en las funciones del software

Aquí la IA no asiste al desarrollador: es una capacidad del producto que el usuario final usa. Esto cambia los requisitos no funcionales que hay que satisfacer, porque la disponibilidad, la latencia, la privacidad y el costo del modelo pasan a ser atributos de calidad del sistema entregado.

Inferencia local frente a inferencia en la nube

Criterio	Modelo local (Ollama)	Modelo en la nube (OpenAI)
Disponibilidad	Funciona sin internet. Es la única opción que satisface el requisito de operación local del enunciado.	Requiere conexión permanente.
Privacidad	Los datos no salen de la máquina.	Los datos se envían a un tercero.
Costo	Sin costo por uso; costo de hardware.	Costo por token consumido.
Capacidad	Menor: un modelo de 7 a 11 mil millones de parámetros interpreta bien instrucciones estructuradas, pero es más frágil ante frases ambiguas.	Mayor precisión, especialmente en visión y en instrucciones complejas.
Latencia	Depende del hardware; sin latencia de red.	Latencia de red más tiempo de inferencia.

Requisitos	Memoria y preferentemente GPU en la máquina del usuario.	Una clave de API.
------------	--	-------------------

La estrategia adoptada: híbrida con degradación elegante

En lugar de elegir una de las dos opciones, la capa de IA define una interfaz única con tres estrategias configurables: solo local, solo nube, o híbrida. En modo híbrido se intenta primero el modelo local y, si no está disponible o falla, se recurre automáticamente al de la nube.

La justificación es doble. Por un lado cumple el requisito del enunciado de contar con IA local en el escritorio y en el móvil, con el reconocimiento de voz ejecutándose en el dispositivo. Por otro lado es una decisión de ingeniería sobre un riesgo concreto: si el día de la evaluación el modelo local no arranca —por memoria, por un servicio detenido, por un modelo no descargado—, la demostración no se cae, porque el sistema degrada al proveedor de la nube y lo informa en la interfaz. La interfaz muestra en todo momento qué proveedor respondió, de modo que la degradación es visible y no silenciosa.

El reconocimiento de voz

El reconocimiento de voz se ejecuta en el dispositivo mediante la Web Speech API, que en escritorio y en Android utiliza el motor de voz del sistema operativo. La aplicación local se encarga por lo tanto de la conversión de voz a texto y de la entrega textual del contexto, y es la capa de IA la que interpreta ese texto. Esta separación es la que permite que el módulo de voz del móvil y el del escritorio compartan exactamente la misma lógica de interpretación.

Funciones del producto que usan IA

- **Asistente de modelado:** traduce una instrucción hablada o escrita en operaciones sobre el diagrama, con el contexto del diagrama actual para poder referirse a clases existentes.
- **Reconocimiento de diagramas desde imagen:** usa un modelo con capacidad de visión para transcribir un diagrama fotografiado a clases, atributos y relaciones.
- **Asistente de voz móvil:** la interfaz completa de la aplicación móvil, sin controles gráficos.
- **Guía de usuario como agente:** responde preguntas sobre la herramienta, con la base de conocimiento escrita como contexto obligatorio.

[DIAGRAMA 04] Diagrama de secuencia de la capa de IA en modo híbrido

Qué debe mostrar: El recorrido completo de una instrucción por voz: Usuario → PWA/Escritorio (reconocimiento de voz en el dispositivo) → Servidor /api/ai/uml-actions → intento contra Ollama local → en caso de fallo, llamada a OpenAI → devolución de las acciones en JSON → aplicación como operaciones granulares → difusión por WebSocket al resto de los participantes. Mostrar el camino alternativo de fallo con un fragmento "alt".

Cómo construirlo: Diagrama de secuencia UML con los participantes en columnas y un bloque combinado "alt" para la bifurcación entre IA local e IA en la nube.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 4: Diagrama de secuencia de la capa de IA en modo híbrido".

9. Spring Boot y descripción de las librerías utilizadas

Spring Boot es un proyecto del ecosistema Spring que elimina la configuración manual necesaria para levantar una aplicación Java empresarial. Su aporte se apoya en tres mecanismos: la autoconfiguración, que deduce la configuración a partir de las dependencias presentes en el classpath; los starters, que agrupan dependencias compatibles entre sí bajo un único artefacto; y el servidor embebido, que permite empaquetar la aplicación como un archivo ejecutable autónomo.

Se eligió Spring Boot como destino de la generación por tres razones. Primero, porque su modelo de anotaciones hace que la correspondencia entre el diagrama de clases y el código sea directa y legible: una clase UML es una clase anotada con `@Entity`, una relación es una anotación de asociación. Segundo, porque el patrón en capas entidad-repositorio-servicio-controlador es un estándar reconocido, lo que hace que el código generado le resulte familiar a cualquier desarrollador Java. Tercero, porque Spring Data JPA genera las operaciones CRUD a partir de la declaración de una interfaz, lo que reduce sustancialmente el volumen de código a generar.

9.1 Librerías del backend generado

El proyecto que la herramienta genera es un proyecto Maven con Spring Boot 3.3.3 sobre Java 17. Sus dependencias, y la razón de cada una:

Dependencia	Función	Por qué se incluye
spring-boot-starter-parent 3.3.3	POM padre que fija las versiones compatibles de todo el ecosistema.	Evita conflictos de versiones entre las dependencias de Spring.
spring-boot-starter-web	Spring MVC, Jackson para JSON y el servidor Tomcat embebido.	Es lo que permite exponer los controladores REST generados y ejecutar la aplicación sin instalar un servidor aparte.
spring-boot-starter-data-jpa	Spring Data JPA con Hibernate como implementación de JPA.	Traduce las entidades anotadas a tablas y genera las operaciones CRUD a partir de la interfaz <code>JpaRepository</code> .
postgresql (driver JDBC)	Controlador de conexión a PostgreSQL.	Se incluye cuando se elige la opción de base de datos PostgreSQL.
h2	Base de datos relacional embebida en memoria.	Se incluye en la opción alternativa, que permite ejecutar el backend generado sin instalar ninguna base de datos: útil para una demostración inmediata.
lombok	Generación de getters, setters y constructores por anotación.	Reduce el código repetitivo de las entidades generadas, dejándolas legibles.
springdoc-openapi-starter-webmvc-ui 2.2.0	Documentación OpenAPI y interfaz Swagger UI generadas automáticamente.	Permite explorar y probar los endpoints generados desde el navegador, sin herramienta

		externa.
spring-boot-starter-test	JUnit 5, Mockito y utilidades de prueba de Spring.	Deja el proyecto generado preparado para incorporar pruebas automatizadas.
spring-boot-maven-plugin	Empaquetado del JAR ejecutable y ejecución desde Maven.	Permite levantar el proyecto con un solo comando.

Cuando se elige la opción de PostgreSQL, la generación incluye además un archivo docker-compose y un Dockerfile, de modo que la base de datos se levante con un comando y no haya que instalarla manualmente.

Anotaciones JPA que emite el generador

Anotación	Dónde se emite	Origen en el diagrama
@Entity, @Table	En cada clase generada.	Regla R1: una clase es una tabla.
@Id, @GeneratedValue	En la columna identificadora.	Clave primaria subrogada de la regla R1.
@Column	En cada atributo.	Regla R2: atributo a columna.
@JoinColumn, @ForeignKey	En las columnas de relación.	Reglas R3 a R5: implementación de las asociaciones.
ON DELETE CASCADE en la definición de la foránea	En composiciones y en la herencia.	Reglas R6 y R8: dependencia existencial.
@RestController, @RequestMapping, @GetMapping y equivalentes	En los controladores.	Un controlador con las cinco operaciones CRUD por cada clase del diagrama.
@Service, @Repository, @Autowired	En las capas de servicio y persistencia.	Estructura en capas del desarrollo basado en componentes.

9.2 Librerías del servidor de la herramienta

El servidor propio de la herramienta —que no debe confundirse con el backend que la herramienta genera— está construido sobre Node.js con TypeScript. Sus dependencias:

Librería	Función	Por qué se eligió
express	Servidor HTTP y enrutamiento.	Estándar de hecho en Node, mínimo y suficiente para una API REST de este tamaño.
ws	Servidor WebSocket.	Implementación ligera del protocolo, sin la capa de abstracción y los reintentos propios de alternativas más grandes, que aquí se implementan a medida.
pg	Cliente de PostgreSQL con pool de conexiones.	Acceso directo a SQL, que es lo que se necesita: las consultas son pocas y deben ser

		transaccionales y explícitas.
multer	Recepción de archivos subidos.	Maneja la carga de las imágenes de diagramas, reemplazando el servicio de almacenamiento externo.
cors	Control de origen cruzado.	Necesario para que el cliente y la PWA del teléfono puedan consumir la API desde otro origen en la red local.
dotenv	Carga de variables de entorno.	Mantiene las credenciales fuera del código fuente.
uuid	Generación de identificadores únicos.	Produce los identificadores de operación que hacen idempotente la sincronización.
tsx / typescript	Ejecución y compilación de TypeScript.	Tipado estático en servidor y cliente, con los tipos del protocolo compartidos entre ambos.

9.3 Librerías del cliente

Librería	Función	Por qué se eligió
react 18 + typescript	Biblioteca de interfaz y tipado.	Modelo declarativo adecuado para una interfaz que refleja un estado que cambia por eventos remotos.
vite	Servidor de desarrollo y empaquetador.	Arranque y recarga rápidos; soporte nativo de variables de entorno y de PWA.
reactflow	Lienzo de nodos y aristas.	Resuelve el arrastre, el zoom y el desplazamiento del lienzo, que no son el problema que este proyecto debe resolver.
zustand	Gestión de estado global.	API mínima y sin envoltorios; permite leer y escribir el estado desde fuera de los componentes, lo que era necesario para aplicar las operaciones que llegan del servidor.
react-router-dom	Enrutamiento.	Separa las cinco vistas: pizarra, diseño de datos, guía, voz y diagnóstico.
@mui/material	Componentes de interfaz.	Se conserva del sistema base; el rediseño posterior se hizo con un sistema de tokens propio.
jszip + file-saver	Creación y descarga de archivos ZIP.	Empaquetan el proyecto Spring Boot generado para su descarga desde el navegador.

uuid	Identificadores únicos.	Identificadores de operación y de elementos del diagrama.
------	-------------------------	---

9.4 Estructura del proyecto generado

Cada generación produce un proyecto Maven completo con la siguiente estructura, donde <Clase> representa cada clase del diagrama de origen:

```

spring-crud/
├── pom.xml
├── docker-compose.yml          (solo en la opción PostgreSQL)
├── Dockerfile                 (solo en la opción PostgreSQL)
├── README.md
└── src/main/
    ├── java/com/example/demo/
    │   ├── DemoApplication.java    clase principal
    │   ├── model/<Clase>.java      entidad JPA
    │   ├── repository/<Clase>Repository.java
    │   ├── service/<Clase>Service.java
    │   └── controller/<Clase>Controller.java
    ├── resources/
    └── application.properties

```

Parte II – Registro del proceso de desarrollo (PUDS)

Esta parte documenta el recorrido de los cinco flujos de trabajo del Proceso Unificado sobre las tres iteraciones del proyecto.

1. Flujo de trabajo: Captura de Requisitos

1.1 Actores

El sistema reconoce dos actores humanos y dos actores del sistema. Es importante señalar desde el principio que la distinción entre anfitrión y participante es funcional y no de permisos: el sistema no implementa por el momento autenticación ni control de accesos diferenciado, de modo que todo participante de una pizarra tiene las mismas capacidades de edición. Esta limitación se declara en el apartado correspondiente y se documenta aquí para que los casos de uso no describan un comportamiento que el software no tiene.

Actor	Tipo	Descripción y responsabilidades
Modelador anfitrión	Humano, primario	Es quien crea la pizarra y comparte el enlace de invitación. Modela el diagrama de clases, dicta instrucciones al asistente, importa diagramas desde imagen o archivo, ejecuta el módulo de diseño de datos y genera el backend, el frontend de prueba y la colección de Postman. Administra las pizarras: crear, renombrar y eliminar.
Modelador participante	Humano, primario	Se une a una pizarra existente mediante el enlace. Tiene las mismas capacidades de modelado y de uso del asistente. Su identidad visible —nombre y color— se elige en el cliente y se muestra al resto en el indicador de presencia.
Proveedor de IA	Sistema, secundario	Servicio de inferencia que interpreta las instrucciones en lenguaje natural y reconoce los diagramas de las imágenes. Puede ser el modelo local (Ollama) o el de la nube (OpenAI); el sistema decide cuál según la estrategia configurada.
Motor de voz del dispositivo	Sistema, secundario	Servicio de reconocimiento y síntesis de voz del sistema operativo, accedido a través del navegador. Convierte la voz del

		usuario en texto y la respuesta del sistema en voz.
--	--	---

[DIAGRAMA 05] Diagrama de actores y su relación con el sistema

Qué debe mostrar: Los cuatro actores alrededor del límite del sistema, con los actores humanos a la izquierda y los actores del sistema a la derecha. Indicar que el Modelador participante hereda del Modelador anfitrión en cuanto a capacidades de modelado (generalización de actores), señalando con una nota que la diferencia es funcional y no de permisos.

Cómo construirlo: Diagrama de casos de uso reducido a actores, con el rectángulo del sistema vacío. Usar el triángulo de generalización entre los dos actores humanos.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 5: Diagrama de actores y su relación con el sistema".

1.2 Casos de uso

ID	Nombre	Actor
CU01	Crear pizarra colaborativa	Anfitrión
CU02	Compartir enlace de invitación	Anfitrión
CU03	Unirse a una pizarra existente	Participante
CU04	Administrar pizarras (renombrar y eliminar)	Anfitrión
CU05	Crear y editar clases con sus atributos	Anfitrión, Participante
CU06	Crear relaciones entre clases	Anfitrión, Participante
CU07	Crear relación muchos a muchos con clase asociativa	Anfitrión, Participante
CU08	Editar el diagrama mediante instrucción escrita	Anfitrión, Participante
CU09	Editar el diagrama mediante instrucción por voz	Anfitrión, Participante
CU10	Operar el sistema por voz desde el teléfono	Anfitrión, Participante
CU11	Importar un diagrama desde una fotografía o boceto	Anfitrión, Participante
CU12	Importar un diagrama desde archivo (JSON o XMI)	Anfitrión, Participante
CU13	Exportar el diagrama a otra herramienta CASE	Anfitrión, Participante
CU14	Visualizar los participantes conectados y su actividad	Anfitrión, Participante
CU15	Trabajar sin conexión y sincronizar al reconectar	Anfitrión, Participante
CU16	Consultar la bitácora de operaciones del diagrama	Anfitrión
CU17	Generar el modelo conceptual de	Anfitrión, Participante

	datos	
CU18	Generar el mapeo objeto-relacional y el esquema lógico	Anfitrión, Participante
CU19	Analizar la normalización del esquema	Anfitrión, Participante
CU20	Generar el DDL de la base de datos	Anfitrión, Participante
CU21	Generar el backend en Spring Boot	Anfitrión
CU22	Generar la colección de pruebas para Postman	Anfitrión
CU23	Generar el frontend de prueba	Anfitrión
CU24	Consultar la guía de usuario	Anfitrión, Participante
CU25	Consultar el diagnóstico del sistema y el estado de la IA	Anfitrión

1.3 Priorizar casos de uso

La priorización combina el valor para el usuario con el riesgo técnico. Los casos de uso de riesgo alto se adelantaron a la primera iteración, siguiendo el principio de PUDS de atacar primero lo que puede invalidar la arquitectura.

ID	Nombre	Actor	Estado	Prioridad	Riesgo
CU01	Crear pizarra colaborativa	Anfitrión	Completado	Alta	Bajo
CU02	Compartir enlace de invitación	Anfitrión	Completado	Media	Bajo
CU03	Unirse a una pizarra existente	Participante	Completado	Alta	Medio
CU04	Administrar pizarras (renombrar y eliminar)	Anfitrión	Completado	Media	Bajo
CU05	Crear y editar clases con sus atributos	Anfitrión, Participante	Completado	Alta	Medio
CU06	Crear relaciones entre clases	Anfitrión, Participante	Completado	Alta	Medio
CU07	Crear relación muchos a muchos con clase asociativa	Anfitrión, Participante	Completado	Alta	Alto
CU08	Editar el diagrama mediante instrucción escrita	Anfitrión, Participante	Completado	Alta	Alto
CU09	Editar el diagrama mediante	Anfitrión, Participante	Completado	Alta	Alto

	instrucción por voz				
CU10	Operar el sistema por voz desde el teléfono	Anfitrión, Participante	Completado	Alta	Alto
CU11	Importar un diagrama desde una fotografía o boceto	Anfitrión, Participante	Completado	Alta	Alto
CU12	Importar un diagrama desde archivo (JSON o XMI)	Anfitrión, Participante	Completado	Media	Medio
CU13	Exportar el diagrama a otra herramienta CASE	Anfitrión, Participante	Parcial	Media	Medio
CU14	Visualizar los participantes conectados y su actividad	Anfitrión, Participante	Completado	Alta	Medio
CU15	Trabajar sin conexión y sincronizar al reconectar	Anfitrión, Participante	Completado	Alta	Alto
CU16	Consultar la bitácora de operaciones del diagrama	Anfitrión	Completado	Media	Bajo
CU17	Generar el modelo conceptual de datos	Anfitrión, Participante	Completado	Alta	Medio
CU18	Generar el mapeo objeto-relacional y el esquema lógico	Anfitrión, Participante	Completado	Alta	Alto
CU19	Analizar la normalización del esquema	Anfitrión, Participante	Completado	Alta	Medio
CU20	Generar el DDL de la base de datos	Anfitrión, Participante	Completado	Alta	Medio
CU21	Generar el backend en Spring Boot	Anfitrión	Completado	Alta	Medio
CU22	Generar la colección de pruebas para Postman	Anfitrión	Completado	Media	Bajo
CU23	Generar el frontend de prueba	Anfitrión	Completado	Baja	Bajo
CU24	Consultar la guía de usuario	Anfitrión, Participante	Completado	Media	Bajo

CU25	Consultar el diagnóstico del sistema y el estado de la IA	Anfitrión	Completado	Media	Bajo
------	---	-----------	------------	-------	------

El único caso de uso con estado parcial es CU13: la exportación funciona en XMI 2.5 y en el formato JSON propio, pero el adaptador para el formato nativo de Architech está especificado y no implementado, por falta de un archivo de ejemplo de ese formato.

Ciclo 1 – Elaboración: arquitectura colaborativa

ID	Nombre	Actor	Estado	Prioridad	Riesgo
CU01	Crear pizarra colaborativa	Anfitrión	Completado	Alta	Bajo
CU02	Compartir enlace de invitación	Anfitrión	Completado	Media	Bajo
CU03	Unirse a una pizarra existente	Participante	Completado	Alta	Medio
CU04	Administrar pizarras (renombrar y eliminar)	Anfitrión	Completado	Media	Bajo
CU05	Crear y editar clases con sus atributos	Anfitrión, Participante	Completado	Alta	Medio
CU06	Crear relaciones entre clases	Anfitrión, Participante	Completado	Alta	Medio
CU07	Crear relación muchos a muchos con clase asociativa	Anfitrión, Participante	Completado	Alta	Alto
CU14	Visualizar los participantes conectados y su actividad	Anfitrión, Participante	Completado	Alta	Medio
CU15	Trabajar sin conexión y sincronizar al reconectar	Anfitrión, Participante	Completado	Alta	Alto
CU16	Consultar la bitácora de operaciones del diagrama	Anfitrión	Completado	Media	Bajo

Ciclo 2 – Construcción: interfaz asistida por IA

ID	Nombre	Actor	Estado	Prioridad	Riesgo
CU08	Editar el diagrama mediante instrucción escrita	Anfitrión, Participante	Completado	Alta	Alto
CU09	Editar el	Anfitrión,	Completado	Alta	Alto

	diagrama mediante instrucción por voz	Participante			
CU10	Operar el sistema por voz desde el teléfono	Anfitrión, Participante	Completado	Alta	Alto
CU11	Importar un diagrama desde una fotografía o boceto	Anfitrión, Participante	Completado	Alta	Alto
CU24	Consultar la guía de usuario	Anfitrión, Participante	Completado	Media	Bajo
CU25	Consultar el diagnóstico del sistema y el estado de la IA	Anfitrión	Completado	Media	Bajo

Ciclo 3 – Construcción: diseño de datos y generación

ID	Nombre	Actor	Estado	Prioridad	Riesgo
CU12	Importar un diagrama desde archivo (JSON o XMI)	Anfitrión, Participante	Completado	Media	Medio
CU13	Exportar el diagrama a otra herramienta CASE	Anfitrión, Participante	Parcial	Media	Medio
CU17	Generar el modelo conceptual de datos	Anfitrión, Participante	Completado	Alta	Medio
CU18	Generar el mapeo objeto-relacional y el esquema lógico	Anfitrión, Participante	Completado	Alta	Alto
CU19	Analizar la normalización del esquema	Anfitrión, Participante	Completado	Alta	Medio
CU20	Generar el DDL de la base de datos	Anfitrión, Participante	Completado	Alta	Medio
CU21	Generar el backend en Spring Boot	Anfitrión	Completado	Alta	Medio
CU22	Generar la colección de pruebas para Postman	Anfitrión	Completado	Media	Bajo
CU23	Generar el frontend de prueba	Anfitrión	Completado	Baja	Bajo

1.4 Detallar casos de uso

Cada ficha describe el caso de uso con su propósito, actores, precondition, flujo principal, postcondición y excepciones. Antes de cada ficha se indica el diagrama de caso de uso individual que corresponde.

CICLO 1

[DIAGRAMA 06] Caso de uso individual CU01 – Crear pizarra colaborativa

Qué debe mostrar: El actor Anfitrión vinculado al caso de uso "Crear pizarra colaborativa". Relaciones a dibujar: Sin relaciones con otros casos de uso.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 6: Caso de uso individual CU01 – Crear pizarra colaborativa".

ID	CU01
Nombre	Crear pizarra colaborativa
Propósito	Disponer de un espacio de trabajo nuevo y vacío sobre el cual modelar un diagrama de clases de forma compartida.
Actor principal	Anfitrión
Actor iniciador	Anfitrión
Precondición	El servidor debe estar en ejecución y accesible desde el cliente.
Flujo principal	1. El usuario abre el menú de la pizarra actual en la barra de herramientas. 2. Selecciona "Nueva pizarra". 3. Ingresa el nombre de la pizarra. 4. El sistema crea el diagrama asociado y la pizarra en una sola transacción. 5. El sistema conmuta a la nueva pizarra y establece la sesión colaborativa sobre ella.
Postcondición	Existe una pizarra nueva con su diagrama vacío, registrada en la base de datos, y el cliente está conectado a ella.
Excepciones	Si el servidor no responde, la pizarra no se crea y el sistema informa la falla; la lista de pizarras conocida se conserva en caché local.

[DIAGRAMA 07] Caso de uso individual CU02 – Compartir enlace de invitación

Qué debe mostrar: El actor Anfitrión vinculado al caso de uso "Compartir enlace de invitación".

Relaciones a dibujar: Sin relaciones. Requiere una pizarra creada por CU01.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 7: Caso de uso individual CU02 – Compartir enlace de invitación".

ID	CU02
Nombre	Compartir enlace de invitación
Propósito	Permitir que otras personas se incorporen a la misma pizarra.
Actor principal	Anfitrión
Actor iniciador	Anfitrión
Precondición	Debe existir una pizarra activa.
Flujo principal	1. El usuario abre el menú de la pizarra. 2. Selecciona "Copiar enlace para invitar". 3. El sistema copia al portapapeles la dirección de la pizarra. 4. El usuario comparte esa dirección por el medio que elija.
Postcondición	El enlace queda en el portapapeles y cualquiera que lo abra accede a la misma pizarra.
Excepciones	Si el navegador deniega el acceso al portapapeles, el sistema muestra la dirección para copiarla manualmente.

[DIAGRAMA 08] Caso de uso individual CU03 – Unirse a una pizarra existente

Qué debe mostrar: El actor Participante vinculado al caso de uso "Unirse a una pizarra existente".

Relaciones a dibujar: Sin relaciones. Depende del enlace producido por CU02.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 8: Caso de uso individual CU03 – Unirse a una pizarra existente".

ID	CU03
Nombre	Unirse a una pizarra existente
Propósito	Incorporarse a una sesión de modelado en curso y recibir el estado actual del diagrama.

Actor principal	Participante
Actor iniciador	Participante
Precondición	El participante debe disponer del enlace de la pizarra y el servidor debe estar accesible.
Flujo principal	1. El participante abre el enlace recibido. 2. El cliente genera o recupera su identidad local: identificador, nombre y color. 3. El cliente abre la conexión WebSocket y envía el mensaje de incorporación con el último número de secuencia que tenga aplicado. 4. El servidor responde con el documento completo, o solo con las operaciones faltantes si el cliente estaba poco atrasado. 5. El servidor difunde la lista actualizada de participantes al resto.
Postcondición	El participante ve el diagrama en su estado actual y aparece en el indicador de presencia de los demás.
Excepciones	Si la conexión falla, el cliente muestra el documento almacenado en caché local y reintenta la conexión con espera creciente.

[DIAGRAMA 09] Caso de uso individual CU04 – Administrar pizarras

Qué debe mostrar: El actor Anfitrión vinculado al caso de uso "Administrar pizarras". Relaciones a dibujar: Sin relaciones.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 9: Caso de uso individual CU04 – Administrar pizarras".

ID	CU04
Nombre	Administrar pizarras
Propósito	Renombrar o eliminar pizarras para mantener ordenado el espacio de trabajo.
Actor principal	Anfitrión
Actor iniciador	Anfitrión
Precondición	Debe existir al menos una pizarra.
Flujo principal	1. El usuario abre el menú de pizarras. 2. Elige renombrar o eliminar sobre la pizarra deseada. 3. En el caso de renombrar, ingresa el nuevo

	nombre. 4. En el caso de eliminar, confirma la acción. 5. El sistema aplica el cambio y actualiza la lista.
Postcondición	La pizarra queda renombrada, o eliminada junto con su diagrama y su bitácora de operaciones por borrado en cascada.
Excepciones	La última pizarra no se puede eliminar. Si el servidor no responde, la operación no se aplica y se informa.

[DIAGRAMA 10] Caso de uso individual CU05 – Crear y editar clases con sus atributos

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Crear y editar clases con sus atributos". Relaciones a dibujar: Es incluido por CU08, CU09, CU10, CU11 y CU12. Extendido por CU15 (trabajo sin conexión).

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 10: Caso de uso individual CU05 – Crear y editar clases con sus atributos".

ID	CU05
Nombre	Crear y editar clases con sus atributos
Propósito	Construir el modelo de clases: agregar clases, renombrarlas, y agregar, modificar o eliminar sus atributos con tipo y visibilidad.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Debe haber una pizarra activa.
Flujo principal	1. El usuario presiona el botón de nueva clase, o lo hace sobre una clase existente para editarla. 2. Escribe el nombre de la clase directamente sobre ella. 3. Agrega atributos indicando nombre, tipo y visibilidad. 4. El sistema aplica el cambio de inmediato en la vista local. 5. El sistema persiste la operación correspondiente en el dispositivo y la envía al servidor. 6. El servidor le asigna un número de secuencia, la persiste y la difunde al resto de los participantes.
Postcondición	La clase y sus atributos forman parte del documento compartido y son visibles para todos los participantes.

Excepciones	Si el servidor rechaza la operación —por ejemplo porque otro participante eliminó esa clase—, el cliente detecta el rechazo y solicita el documento completo para resincronizar.
--------------------	--

[DIAGRAMA 11] Caso de uso individual CU06 – Crear relaciones entre clases

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Crear relaciones entre clases". Relaciones a dibujar: Es incluido por CU08, CU09, CU10, CU11 y CU12. Extendido por CU15.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 11: Caso de uso individual CU06 – Crear relaciones entre clases".

ID	CU06
Nombre	Crear relaciones entre clases
Propósito	Establecer las relaciones estructurales del modelo con su tipo y sus multiplicidades.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Deben existir al menos dos clases en el diagrama.
Flujo principal	1. El usuario inicia el modo relación desde la clase de origen. 2. Selecciona el tipo de relación: asociación, agregación, composición, herencia o dependencia. 3. Indica las multiplicidades de cada extremo. 4. Hace clic en la clase de destino. 5. El sistema crea la relación y la difunde como operación.
Postcondición	La relación existe en el documento compartido con su tipo y multiplicidades, y queda disponible para el mapeo relacional y la generación de código.
Excepciones	El sistema rechaza una relación cuyos extremos ya no existen, y no permite relacionar una clase consigo misma.

[DIAGRAMA 12] Caso de uso individual CU07 – Crear relación muchos a muchos con clase asociativa

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Crear relación muchos a muchos con clase asociativa". Relaciones a dibujar: Incluye a CU05 (crea la clase asociativa) y a CU06 (crea las dos relaciones). Extendido por CU15.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y

su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 12: Caso de uso individual CU07 – Crear relación muchos a muchos con clase asociativa".

ID	CU07
Nombre	Crear relación muchos a muchos con clase asociativa
Propósito	Resolver una relación de muchos a muchos generando la clase asociativa intermedia que el modelo relacional requiere.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Deben existir las dos clases a relacionar.
Flujo principal	1. El usuario activa el modo muchos a muchos desde el menú de modelado. 2. Selecciona la primera clase. 3. Selecciona la segunda clase. 4. El sistema crea una clase intermedia marcada como asociativa, que referencia a ambas. 5. El sistema crea las dos relaciones de asociación con multiplicidad uno a muchos desde cada clase hacia la intermedia. 6. El usuario puede agregar atributos propios a la clase asociativa.
Postcondición	El diagrama contiene la clase asociativa y sus dos relaciones, de modo que el mapeo produce la tabla intermedia con clave primaria compuesta.
Excepciones	Si alguna de las dos clases se elimina durante la operación, el modo se cancela sin crear nada.

[DIAGRAMA 13] Caso de uso individual CU14 – Visualizar los participantes conectados y su actividad

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Visualizar los participantes conectados y su actividad". Relaciones a dibujar: Sin relaciones. Se activa junto con CU03.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 13: Caso de uso individual CU14 – Visualizar los participantes conectados y su actividad".

ID	CU14
Nombre	Visualizar los participantes conectados y su actividad
Propósito	Saber quién está trabajando en la pizarra y sobre qué elemento, para coordinar el trabajo y evitar interferencias.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Debe haber una sesión colaborativa establecida.
Flujo principal	1. El cliente informa al servidor su identidad al incorporarse, y su selección actual cuando cambia. 2. El servidor mantiene la lista de sesiones por diagrama y la difunde ante cada cambio. 3. La barra muestra el número de participantes y, al abrir el indicador, el nombre, el color y la actividad de cada uno. 4. El servidor depura periódicamente las conexiones muertas para que la lista no conserve participantes fantasma.
Postcondición	Todos los participantes ven la misma lista de presentes y su actividad.
Excepciones	Una misma persona con dos pestañas abiertas se muestra una sola vez, porque la presencia se agrupa por identificador de cliente.

[DIAGRAMA 14] Caso de uso individual CU15 – Trabajar sin conexión y sincronizar al reconectar

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Trabajar sin conexión y sincronizar al reconectar". Relaciones a dibujar: Extiende a CU05, CU06 y CU07: es el comportamiento alternativo ante pérdida de red.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 14: Caso de uso individual CU15 – Trabajar sin conexión y sincronizar al reconectar".

ID	CU15
Nombre	Trabajar sin conexión y sincronizar al reconectar
Propósito	Continuar modelando cuando la red se corta, sin perder trabajo, y reconciliar automáticamente al recuperarla.
Actor principal	Anfitrión, Participante

Actor iniciador	Anfitrión
Precondición	El usuario debe haber abierto la pizarra al menos una vez, para disponer del documento en caché.
Flujo principal	1. El cliente detecta la pérdida de conexión y lo indica en la interfaz. 2. Cada operación del usuario se aplica localmente y se persiste en el almacenamiento del navegador antes de intentar enviarse. 3. La interfaz muestra la cantidad de operaciones en cola. 4. Al recuperar la red, el cliente reenvía la cola completa. 5. El servidor descarta las operaciones que ya tenía registradas, por su identificador único, y acepta las nuevas. 6. El cliente solicita las operaciones que se perdió mientras estuvo desconectado y las aplica.
Postcondición	El trabajo realizado sin conexión forma parte del documento compartido, sin duplicados, y el cliente está al día con el resto.
Excepciones	Si una operación en cola se refiere a un elemento que otro participante eliminó mientras el cliente estaba desconectado, el servidor la rechaza y el cliente resincroniza el documento completo.

[DIAGRAMA 15] Caso de uso individual CU16 – Consultar la bitácora de operaciones del diagrama

Qué debe mostrar: El actor Anfitrión vinculado al caso de uso "Consultar la bitácora de operaciones del diagrama". Relaciones a dibujar: Sin relaciones. Consume lo que registran CU05, CU06 y CU07.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 15: Caso de uso individual CU16 – Consultar la bitácora de operaciones del diagrama".

ID	CU16
Nombre	Consultar la bitácora de operaciones del diagrama
Propósito	Auditar la historia del diagrama: qué cambió, en qué orden y quién lo hizo.
Actor principal	Anfitrión
Actor iniciador	Anfitrión
Precondición	El diagrama debe existir.

Flujo principal	1. Se solicita la bitácora del diagrama al servidor, opcionalmente a partir de un número de secuencia. 2. El servidor devuelve las operaciones ordenadas con su autor, tipo, contenido y marca de tiempo.
Postcondición	Se obtiene la secuencia completa de cambios, utilizable para auditoría y para la defensa de autoría.
Excepciones	La consulta está acotada en cantidad de registros para no producir respuestas desmedidas en diagramas con historia larga.

CICLO 2

[DIAGRAMA 16] Caso de uso individual CU08 – Editar el diagrama mediante instrucción escrita

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Editar el diagrama mediante instrucción escrita". Relaciones a dibujar: Incluye a CU05 y CU06. Es extendido por CU09 (la misma instrucción, dictada).

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 16: Caso de uso individual CU08 – Editar el diagrama mediante instrucción escrita".

ID	CU08
Nombre	Editar el diagrama mediante instrucción escrita
Propósito	Modificar el diagrama describiendo el cambio en lenguaje natural, sin usar los controles gráficos.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Debe haber una pizarra activa y al menos un proveedor de IA disponible.
Flujo principal	1. El usuario abre el asistente desde el menú de modelado. 2. Escribe la instrucción, por ejemplo "creá una clase Mascota con nombre texto y edad entero". 3. El cliente envía la instrucción al servidor junto con el contexto del diagrama actual. 4. El servidor consulta al proveedor de IA según la estrategia configurada y obtiene una lista de acciones en formato JSON. 5. El cliente valida cada acción, resuelve las referencias a clases existentes y la traduce en operaciones granulares.

	6. El sistema aplica las operaciones y las difunde.
Postcondición	El diagrama refleja la instrucción y el cambio es visible para todos los participantes. La interfaz indica qué proveedor de IA respondió.
Excepciones	Si ningún proveedor está disponible, el sistema informa la causa y no modifica el diagrama. Si la respuesta de la IA no es un JSON válido, se descarta y se informa.

[DIAGRAMA 17] Caso de uso individual CU09 – Editar el diagrama mediante instrucción por voz

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Editar el diagrama mediante instrucción por voz". Relaciones a dibujar: Extiende a CU08: solo cambia la forma de ingresar la instrucción.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 17: Caso de uso individual CU09 – Editar el diagrama mediante instrucción por voz".

ID	CU09
Nombre	Editar el diagrama mediante instrucción por voz
Propósito	Dictar la modificación en lugar de escribirla, manteniendo el mismo resultado.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	El navegador debe disponer de reconocimiento de voz y debe haber permiso de micrófono.
Flujo principal	1. El usuario abre el asistente y activa el dictado. 2. El motor de voz del dispositivo transcribe el audio a texto de forma local. 3. El texto transcrito se muestra en la interfaz para su verificación. 4. A partir de allí el flujo continúa como en CU08.
Postcondición	El diagrama refleja la instrucción dictada.
Excepciones	Si se deniega el permiso de micrófono, el sistema lo informa y ofrece la entrada por texto. Si no se detecta voz, se informa y se permite reintentar.

[DIAGRAMA 18] Caso de uso individual CU10 – Operar el sistema por voz desde el teléfono

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Operar el sistema por voz

desde el teléfono". Relaciones a dibujar: Incluye a CU05, CU06 y CU08. Participa el actor Motor de voz del dispositivo.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 18: Caso de uso individual CU10 – Operar el sistema por voz desde el teléfono".

ID	CU10
Nombre	Operar el sistema por voz desde el teléfono
Propósito	Modelar sobre la pizarra compartida desde un teléfono, sin interfaz gráfica, usando únicamente la voz.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	El teléfono debe estar en la misma red que el servidor y el navegador debe permitir el micrófono.
Flujo principal	1. El usuario abre la aplicación móvil instalada, que muestra un único control. 2. Toca el control y dicta la instrucción. 3. El reconocimiento de voz se ejecuta en el dispositivo. 4. El sistema interpreta y aplica el cambio en la pizarra compartida. 5. El sistema responde hablando el resumen de lo que hizo. 6. Si la instrucción es una consulta de estado, se responde en el dispositivo sin consultar a la IA.
Postcondición	El diagrama compartido queda modificado y el usuario recibe la confirmación por voz.
Excepciones	Sin conexión, el cambio se guarda en la cola local y el sistema lo informa en la respuesta hablada; se sincroniza al recuperar la red.

[DIAGRAMA 19] Caso de uso individual CU11 – Importar un diagrama desde una fotografía o boceto

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Importar un diagrama desde una fotografía o boceto". Relaciones a dibujar: Incluye a CU05 y CU06. Participa el actor Proveedor de IA con capacidad de visión.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 19: Caso de uso individual CU11 – Importar un diagrama desde una fotografía o boceto".

ID	CU11
Nombre	Importar un diagrama desde una fotografía o boceto
Propósito	Recuperar un diagrama que está en una pizarra física, en papel o en una captura de pantalla, convirtiéndolo en un modelo editable.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Debe haber un proveedor de IA con capacidad de visión disponible.
Flujo principal	1. El usuario elige importar desde una foto. 2. Selecciona la imagen. 3. El sistema archiva la imagen en el servidor como evidencia de la entrada analizada. 4. El servidor envía la imagen al modelo con visión y obtiene las clases y relaciones reconocidas. 5. El sistema normaliza los tipos y las multiplicidades, resuelve las clases asociativas y acomoda las clases en el lienzo. 6. El sistema incorpora el resultado al diagrama actual como operaciones.
Postcondición	El diagrama contiene las clases y relaciones reconocidas, editables como cualquier otra.
Excepciones	Si no se reconoce ninguna clase, el sistema informa que la imagen no pudo interpretarse y sugiere una foto más nítida. La imagen archivada se conserva de todos modos.

[DIAGRAMA 20] Caso de uso individual CU24 – Consultar la guía de usuario

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Consultar la guía de usuario". Relaciones a dibujar: Participa el actor Proveedor de IA, solo si la búsqueda local no encuentra respuesta.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 20: Caso de uso individual CU24 – Consultar la guía de usuario".

ID	CU24
Nombre	Consultar la guía de usuario

Propósito	Resolver dudas sobre el uso de la herramienta preguntando en lenguaje natural.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Ninguna. La guía escrita funciona sin servidor y sin IA.
Flujo principal	1. El usuario abre la guía y escribe su pregunta. 2. El sistema busca en la base de conocimiento local por coincidencia de palabras y términos clave. 3. Si encuentra respuesta, la muestra indicando que proviene de la guía escrita. 4. Si no encuentra, consulta a la IA entregándole la misma base como contexto obligatorio. 5. Muestra la respuesta indicando qué proveedor la produjo.
Postcondición	El usuario obtiene la respuesta, y en todo momento sabe si proviene de la guía escrita o de la IA.
Excepciones	Sin IA disponible y sin coincidencia en la guía, el sistema lo informa y remite al listado completo de preguntas frecuentes, que siempre está disponible.

[DIAGRAMA 21] Caso de uso individual CU25 – Consultar el diagnóstico del sistema y el estado de la IA

Qué debe mostrar: El actor Anfitrión vinculado al caso de uso "Consultar el diagnóstico del sistema y el estado de la IA". Relaciones a dibujar: Sin relaciones.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 21: Caso de uso individual CU25 – Consultar el diagnóstico del sistema y el estado de la IA".

ID	CU25
Nombre	Consultar el diagnóstico del sistema y el estado de la IA
Propósito	Determinar rápidamente qué servicio está faltando cuando algo no funciona.
Actor principal	Anfitrión
Actor iniciador	Anfitrión
Precondición	Ninguna.
Flujo principal	1. El usuario abre la vista de diagnóstico. 2. El sistema verifica la disponibilidad del servidor y

	de la base de datos. 3. Consulta el estado de cada proveedor de IA por separado, con su modelo de texto y de visión. 4. Muestra la identidad de la sesión actual y las imágenes archivadas.
Postcondición	El usuario sabe qué componente está caído y qué acción tomar para corregirlo.
Excepciones	Si el servidor no responde, la vista lo indica explícitamente e informa los comandos necesarios para levantar la base de datos y el servidor.

CICLO 3

[DIAGRAMA 22] Caso de uso individual CU12 – Importar un diagrama desde archivo

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Importar un diagrama desde archivo". Relaciones a dibujar: Incluye a CU05 y CU06.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 22: Caso de uso individual CU12 – Importar un diagrama desde archivo".

ID	CU12
Nombre	Importar un diagrama desde archivo
Propósito	Incorporar un diagrama producido en otra herramienta o exportado previamente.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Debe disponerse de un archivo en un formato soportado.
Flujo principal	1. El usuario elige importar desde archivo. 2. Selecciona el archivo. 3. El sistema determina el formato por la extensión y elige el adaptador correspondiente. 4. El adaptador parsea el archivo y produce las clases y relaciones. 5. El sistema incorpora el resultado al diagrama como operaciones.
Postcondición	El diagrama contiene los elementos del archivo importado.
Excepciones	Si el archivo no es válido o el formato no tiene adaptador disponible, se informa el error y no se modifica el diagrama.

[DIAGRAMA 23] Caso de uso individual CU13 – Exportar el diagrama a otra herramienta CASE

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Exportar el diagrama a otra herramienta CASE". Relaciones a dibujar: Generaliza la exportación a los distintos formatos: cada adaptador es una especialización.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 23: Caso de uso individual CU13 – Exportar el diagrama a otra herramienta CASE".

ID	CU13
Nombre	Exportar el diagrama a otra herramienta CASE
Propósito	Llevar el modelo a otra herramienta sin transcribirlo a mano.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	El diagrama debe tener al menos una clase.
Flujo principal	1. El usuario elige el formato de intercambio en el menú. 2. Solicita la exportación. 3. El sistema convierte el diagrama al formato elegido mediante el adaptador correspondiente. 4. El navegador descarga el archivo resultante.
Postcondición	Se obtiene un archivo con el modelo, legible por la herramienta de destino.
Excepciones	Los formatos cuyo adaptador no está implementado se muestran deshabilitados en el selector.

[DIAGRAMA 24] Caso de uso individual CU17 – Generar el modelo conceptual de datos

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Generar el modelo conceptual de datos". Relaciones a dibujar: Es incluido por CU18.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 24: Caso de uso individual CU17 – Generar el modelo conceptual de datos".

ID	CU17
Nombre	Generar el modelo conceptual de datos
Propósito	Obtener el modelo conceptual —entidades, relaciones y cardinalidades— derivado del diagrama de clases.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	El diagrama debe tener al menos una clase.
Flujo principal	1. El usuario abre la vista de diseño de datos. 2. El sistema recorre el diagrama y clasifica cada clase como entidad fuerte, débil o asociativa. 3. Determina la cardinalidad de cada relación a partir de sus multiplicidades. 4. Identifica las especializaciones a partir de las relaciones de herencia. 5. Presenta el modelo conceptual en forma de tablas.
Postcondición	Se dispone del modelo conceptual como paso previo y justificación del mapeo relacional.
Excepciones	Si el diagrama está vacío, la vista informa que debe modelarse al menos una clase.

[DIAGRAMA 25] Caso de uso individual CU18 – Generar el mapeo objeto-relacional y el esquema lógico

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Generar el mapeo objeto-relacional y el esquema lógico". Relaciones a dibujar: Incluye a CU17. Es incluido por CU19 y CU20.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 25: Caso de uso individual CU18 – Generar el mapeo objeto-relacional y el esquema lógico".

ID	CU18
Nombre	Generar el mapeo objeto-relacional y el esquema lógico
Propósito	Transformar el modelo de objetos en un esquema relacional aplicando las reglas de Rumbaugh, dejando constancia de qué regla produjo cada elemento.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión

Precondición	Debe existir el modelo conceptual, es decir, al menos una clase.
Flujo principal	1. El sistema aplica la regla R1 creando una tabla por clase con clave primaria subrogada. 2. Aplica R2 transformando cada atributo en columna con su tipo SQL. 3. Aplica R8 para las herencias y R9 para las clases asociativas. 4. Aplica R3, R4 y R5 según la cardinalidad de cada relación, y R6 o R7 según su semántica de composición o agregación. 5. Presenta el esquema resultante y la tabla de trazabilidad regla por regla.
Postcondición	Se obtiene el esquema lógico completo con sus claves primarias, foráneas y reglas de borrado, y la justificación de cada decisión.
Excepciones	Las relaciones cuyos extremos no existen se ignoran. Las relaciones de dependencia no generan estructura, por definición.

[DIAGRAMA 26] Caso de uso individual CU19 – Analizar la normalización del esquema

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Analizar la normalización del esquema". Relaciones a dibujar: Incluye a CU18.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 26: Caso de uso individual CU19 – Analizar la normalización del esquema".

ID	CU19
Nombre	Analizar la normalización del esquema
Propósito	Verificar el cumplimiento de la primera, segunda y tercera forma normal, y reportar cada incumplimiento con su justificación y su corrección propuesta.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Debe existir el esquema lógico.
Flujo principal	1. Para cada tabla, el sistema evalúa la 1FN buscando grupos repetitivos y atributos presumiblemente multivaluados. 2. Evalúa la 2FN: en tablas con clave compuesta, busca atributos que dependan de solo una parte de

	la clave. 3. Evalúa la 3FN: busca dependencias transitivas y atributos derivados almacenados. 4. Emite cada hallazgo con el problema, su justificación formal y la corrección propuesta. 5. Presenta el resumen y el detalle por tabla.
Postcondición	Se dispone del análisis de normalización documentado, discutible caso por caso.
Excepciones	El análisis se apoya en indicios estructurales, porque un diagrama de clases no declara el conjunto completo de dependencias funcionales; cada observación se presenta justificada para poder aceptarla o descartarla a criterio del diseñador.

[DIAGRAMA 27] Caso de uso individual CU20 – Generar el DDL de la base de datos

Qué debe mostrar: El actor Anfitrión, Participante vinculado al caso de uso "Generar el DDL de la base de datos". Relaciones a dibujar: Incluye a CU18.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 27: Caso de uso individual CU20 – Generar el DDL de la base de datos".

ID	CU20
Nombre	Generar el DDL de la base de datos
Propósito	Obtener el script de creación del esquema, ejecutable directamente sobre el gestor elegido.
Actor principal	Anfitrión, Participante
Actor iniciador	Anfitrión
Precondición	Debe existir el esquema lógico.
Flujo principal	1. El usuario elige el dialecto: PostgreSQL o MySQL. 2. El sistema ordena las tablas de modo que ninguna clave foránea referencie a una tabla aún no creada. 3. Emite las sentencias de creación con sus claves, restricciones, reglas de borrado e índices sobre las foráneas. 4. El usuario copia el script o lo descarga como archivo.
Postcondición	Se obtiene un script SQL ejecutable que crea el esquema completo.

Excepciones	Si existiera un ciclo de referencias entre tablas, el orden se emite de todos modos; las restricciones involucradas deberían agregarse después de la creación.
--------------------	--

[DIAGRAMA 28] Caso de uso individual CU21 – Generar el backend en Spring Boot

Qué debe mostrar: El actor Anfitrión vinculado al caso de uso "Generar el backend en Spring Boot".

Relaciones a dibujar: Es incluido por CU22.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 28: Caso de uso individual CU21 – Generar el backend en Spring Boot".

ID	CU21
Nombre	Generar el backend en Spring Boot
Propósito	Obtener el proyecto del backend funcionando, derivado del diagrama, sin escribirlo a mano.
Actor principal	Anfitrión
Actor iniciador	Anfitrión
Precondición	El diagrama debe tener al menos una clase.
Flujo principal	1. El usuario solicita la generación del backend. 2. Elige la opción de base de datos: H2 embebida o PostgreSQL con Docker. 3. El sistema genera las entidades JPA con sus anotaciones y claves foráneas, los repositorios, los servicios y los controladores REST. 4. Genera el pom.xml, el application.properties, la clase principal y, en la opción PostgreSQL, los archivos de Docker. 5. Empaqueta todo en un archivo ZIP y lo descarga.
Postcondición	Se obtiene un proyecto Maven completo que compila y expone las operaciones CRUD de cada clase del diagrama.
Excepciones	Si el diagrama está vacío, la opción aparece deshabilitada.

[DIAGRAMA 29] Caso de uso individual CU22 – Generar la colección de pruebas para Postman

Qué debe mostrar: El actor Anfitrión vinculado al caso de uso "Generar la colección de pruebas para Postman". Relaciones a dibujar: Incluye a CU21: la colección se deriva de los controladores generados.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 29: Caso de uso individual CU22 – Generar la colección de pruebas para Postman".

ID	CU22
Nombre	Generar la colección de pruebas para Postman
Propósito	Disponer de las peticiones listas para probar los endpoints del backend generado.
Actor principal	Anfitrión
Actor iniciador	Anfitrión
Precondición	Debe haberse generado el backend.
Flujo principal	1. Junto con el backend, el sistema deriva de cada controlador sus rutas y métodos. 2. Construye una colección de Postman con una petición por operación y por entidad. 3. Incluye la colección en el paquete descargado.
Postcondición	Se obtiene una colección importable en Postman que cubre las operaciones CRUD de todas las entidades.
Excepciones	Las peticiones se generan con datos de ejemplo derivados de los tipos de los atributos; los valores concretos deben ajustarse al probar.

[DIAGRAMA 30] Caso de uso individual CU23 – Generar el frontend de prueba

Qué debe mostrar: El actor Anfitrión vinculado al caso de uso "Generar el frontend de prueba".

Relaciones a dibujar: Sin relaciones.

Cómo construirlo: Diagrama de casos de uso: el óvalo del caso de uso dentro del límite del sistema y su actor afuera. Las inclusiones se dibujan con flecha discontinua y el estereotipo «include» apuntando al caso incluido; las extensiones, con «extend» apuntando al caso extendido.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 30: Caso de uso individual CU23 – Generar el frontend de prueba".

ID	CU23
Nombre	Generar el frontend de prueba
Propósito	Contar con una interfaz mínima para ejercitar el backend generado sin depender de una herramienta externa.
Actor principal	Anfitrión

Actor iniciador	Anfitrión
Precondición	El diagrama debe tener al menos una clase.
Flujo principal	1. El usuario solicita la generación del frontend. 2. El sistema genera las vistas de listado y formulario para cada entidad, conectadas a los endpoints del backend. 3. Empaqueta el resultado y lo descarga.
Postcondición	Se obtiene un frontend que permite ejercitar las operaciones CRUD de cada entidad.
Excepciones	El frontend generado es de prueba: no contempla autenticación ni validaciones de negocio.

1.5 Estructurar el modelo de casos de uso

El modelo se estructura agrupando los casos de uso por módulo y explicitando las relaciones de inclusión y extensión entre ellos, para evitar describir dos veces el mismo comportamiento.

Relaciones entre casos de uso

Relación	Casos de uso involucrados	Justificación
«include»	CU08, CU09 y CU10 incluyen a CU05 y CU06.	Toda instrucción interpretada por la IA termina aplicándose como creación o edición de clases y relaciones: el comportamiento es el mismo y no debe duplicarse.
«include»	CU11 y CU12 incluyen a CU05 y CU06.	La importación, cualquiera sea su origen, incorpora el resultado mediante las mismas operaciones de creación.
«include»	CU18 incluye a CU17.	El mapeo relacional se construye sobre el modelo conceptual: no puede ejecutarse sin él.
«include»	CU19 y CU20 incluyen a CU18.	Tanto el análisis de normalización como el DDL operan sobre el esquema lógico ya mapeado.
«include»	CU22 incluye a CU21.	La colección de Postman se deriva de los controladores que produce la generación del backend.
«extend»	CU09 extiende a CU08.	El dictado por voz es una forma alternativa y opcional de proveer la misma instrucción; el resto del flujo es idéntico.
«extend»	CU15 extiende a CU05, CU06 y	El trabajo sin conexión es una

	CU07.	variante de comportamiento que se activa ante una condición excepcional —la pérdida de red— sobre cualquier operación de edición.
Generalización	CU13 generaliza la exportación a los distintos formatos.	Cada formato es una especialización del mismo caso de uso, resuelta por un adaptador distinto.

[DIAGRAMA 31] Diagrama general de casos de uso

Qué debe mostrar: Los veinticinco casos de uso agrupados en paquetes por módulo: Gestión de pizarras y sesión, Modelado UML, Interfaz asistida por IA, Diseño de datos, Generación de software e Interoperabilidad. Los dos actores humanos a la izquierda, los dos actores del sistema a la derecha. Incluir las relaciones «include», «extend» y la generalización de la tabla anterior.

Cómo construirlo: Diagrama de casos de uso UML. Conviene dibujarlo con los paquetes como rectángulos agrupadores para que las veinticinco elipses queden legibles; si no cabe en una página, dividirlo por módulo y presentar además los tres diagramas por ciclo que siguen.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 31: Diagrama general de casos de uso".

1.5.1 Ciclo 1

[DIAGRAMA 32] Casos de uso del Ciclo 1

Qué debe mostrar: Los diez casos de uso del primer ciclo: CU01 a CU07, CU14, CU15 y CU16, con sus actores y relaciones. Corresponde a la arquitectura colaborativa.

Cómo construirlo: Diagrama de casos de uso con el límite del sistema rotulado "Iteración 1 – Elaboración".

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 32: Casos de uso del Ciclo 1".

1.5.2 Ciclo 2

[DIAGRAMA 33] Casos de uso del Ciclo 2

Qué debe mostrar: Los seis casos de uso del segundo ciclo: CU08 a CU11, CU24 y CU25, con la relación «extend» de CU09 sobre CU08 y las inclusiones hacia CU05 y CU06. Corresponde a la interfaz asistida por IA.

Cómo construirlo: Diagrama de casos de uso. Mostrar como actores del sistema al proveedor de IA y al motor de voz del dispositivo.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 33: Casos de uso del Ciclo 2".

1.5.3 Ciclo 3

[DIAGRAMA 34] Casos de uso del Ciclo 3

Qué debe mostrar: Los nueve casos de uso del tercer ciclo: CU12, CU13 y CU17 a CU23, con la cadena de inclusiones CU17 ← CU18 ← CU19/CU20 y CU21 ← CU22. Corresponde al diseño de datos y la generación de software.

Cómo construirlo: Diagrama de casos de uso. La cadena de inclusiones es lo importante de este diagrama: dibujarla de forma que se lea el orden de dependencia.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 34: Casos de uso del Ciclo 3".

2. Flujo de trabajo: Análisis

2.1 Análisis de arquitectura

2.1.1 Identificar paquetes

El sistema se descompone en paquetes con responsabilidad única. La separación no es por tipo de archivo sino por razón de cambio: cada paquete agrupa lo que se modificaría junto ante un mismo requerimiento nuevo.

Paquete	Responsabilidad	Ubicación en el código
Presentación	Vistas y componentes de interfaz; no contiene lógica de dominio.	uml-board/src/pages, uml-board/src/components
Estado del editor	Estado del diagrama en el cliente y traducción de cada acción del usuario a operaciones.	uml-board/src/store/classStore.ts
Colaboración (cliente)	Conexión, reconexión, aplicación optimista, cola offline y presencia.	uml-board/src/lib/collabClient.ts, offlineQueue.ts
Diseño de datos	Modelo conceptual, reglas de mapeo, normalización y DDL. Sin estado ni efectos.	uml-board/src/utills/dataDesign.ts
Generación de código	Producción del backend, del frontend de prueba y de la colección de Postman.	uml-board/src/utills/backendGenerator.ts, frontendGenerator.ts, postmanGenerator.ts
Interoperabilidad	Adaptadores de importación y exportación por formato.	uml-board/src/utills/architech.ts
IA (cliente)	Envío de instrucciones y traducción de las acciones devueltas en operaciones.	uml-board/src/services/aiPromptService.ts, applyActions.ts
Voz	Reconocimiento y síntesis en el dispositivo.	uml-board/src/lib/speech.ts
Colaboración (servidor)	Aplicación de operaciones, orden total, presencia y puesta al día.	server/src/collab.ts, rooms.ts, ws.ts
API REST	Pizarras, diagramas, bitácora, imágenes y endpoints de IA.	server/src/routes/
IA (servidor)	Proveedores de inferencia y prompts.	server/src/ai/
Persistencia	Esquema y acceso a la base de datos.	server/src/db.ts, server/sql/schema.sql

[DIAGRAMA 35] Diagrama de paquetes

Qué debe mostrar: Los doce paquetes con sus dependencias. Reglas que el diagrama debe hacer evidentes: Presentación depende de Estado del editor, pero no al revés; Estado del editor depende de Colaboración (cliente); Diseño de datos y Generación de código dependen únicamente del modelo UML, no del estado ni de la red; Colaboración (servidor) depende de Persistencia; ningún paquete del servidor

depende de paquetes del cliente.

Cómo construirlo: Diagrama de paquetes UML, con los paquetes del cliente agrupados en un paquete contenedor y los del servidor en otro. Dibujar las dependencias con flechas discontinuas. Es importante que no aparezcan dependencias circulares.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 35: Diagrama de paquetes".

2.1.2 Relacionar paquetes y casos de uso

Paquete	Casos de uso que realiza
Presentación	Todos, como capa de interacción.
Estado del editor	CU05, CU06, CU07
Colaboración (cliente y servidor)	CU01, CU03, CU14, CU15, CU16
IA (cliente y servidor)	CU08, CU09, CU10, CU11, CU24
Voz	CU09, CU10
Diseño de datos	CU17, CU18, CU19, CU20
Generación de código	CU21, CU22, CU23
Interoperabilidad	CU12, CU13
API REST	CU01, CU02, CU04, CU16, CU25
Persistencia	CU01, CU04, CU15, CU16

2.2 Análisis de casos de uso

Para los casos de uso de mayor riesgo se identificaron las clases de análisis en las tres categorías habituales: de interfaz, de control y de entidad.

Caso de uso	Clases de interfaz	Clases de control	Clases de entidad
CU05 Editar clases	Pizarra, Nodo de clase	Gestor del estado del editor, Cliente colaborativo	Diagrama, Clase, Atributo, Operación
CU15 Trabajar sin conexión	Indicador de estado y de cola	Cliente colaborativo, Cola de operaciones	Operación, Documento en caché
CU08 Instrucción escrita	Modal del asistente	Servicio de IA, Aplicador de acciones, Proveedor de inferencia	Diagrama, Clase, Relación, Operación
CU18 Mapeo relacional	Vista de diseño de datos	Constructor del modelo conceptual, Mapeador de reglas	Entidad conceptual, Tabla, Columna, Regla aplicada
CU21 Generar backend	Barra de herramientas, Diálogo de opciones	Generador de backend, Empaquetador	Clase, Relación, Archivo generado

[DIAGRAMA 36] Diagrama de clases de análisis de CU15 (trabajo sin conexión)

Qué debe mostrar: Las clases de análisis del caso de uso más riesgoso, con sus estereotipos: «boundary» Indicador de estado; «control» Cliente colaborativo y Cola de operaciones; «entity» Operación y Documento en caché. Mostrar el flujo: la operación se crea, se persiste en la cola, se intenta

enviar, y al confirmarse se retira de la cola.

Cómo construirlo: Diagrama de clases con los estereotipos de análisis de PUDS, usando los íconos de interfaz, control y entidad.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 36: Diagrama de clases de análisis de CU15 (trabajo sin conexión)".

[DIAGRAMA 37] Diagrama de clases de análisis de CU08 (instrucción por IA)

Qué debe mostrar: Las clases de análisis de la interpretación por IA, mostrando que el proveedor de inferencia es una clase de control con dos realizaciones posibles —local y nube— y que el aplicador de acciones es el que valida antes de convertir en operaciones.

Cómo construirlo: Diagrama de clases de análisis con estereotipos.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 37: Diagrama de clases de análisis de CU08 (instrucción por IA)".

2.3 Análisis de paquetes

El análisis de las dependencias entre paquetes arrojó tres reglas que se adoptaron como restricciones de arquitectura, verificables por inspección del código:

1. Ningún paquete de presentación contiene lógica de dominio: las vistas solo leen estado y disparan acciones. Esto es lo que permitió rediseñar la interfaz por completo sin tocar el motor de colaboración.
2. Los paquetes de diseño de datos y de generación de código dependen exclusivamente del modelo UML, y no del estado del editor ni de la red. Por eso son funciones puras, verificables de forma aislada: el DDL generado pudo probarse ejecutándolo contra una base real sin levantar la interfaz.
3. El servidor no depende de ningún paquete del cliente. Los tipos del protocolo están duplicados de forma deliberada en ambos lados, con la advertencia de mantenerlos sincronizados, para no introducir un acoplamiento de compilación entre dos proyectos que se despliegan por separado.

3. Flujo de trabajo: Diseño

3.1 Arquitectura de diseño

3.1.1 Diseño físico (modelo de despliegue)

El sistema se despliega en tres nodos, uno de los cuales es opcional:

Nodo	Artefactos desplegados	Comunicación
Equipo del usuario (escritorio)	Navegador con la aplicación cliente; almacenamiento IndexedDB con la cola de operaciones y la caché del documento.	HTTP y WebSocket hacia el nodo servidor.
Servidor de la aplicación	Proceso Node.js con la API REST, el servidor WebSocket y la capa de IA; directorio de imágenes archivadas.	TCP hacia el nodo de base de datos; HTTP hacia el proveedor de IA en la nube; HTTP hacia el servicio local de inferencia.
Nodo de base de datos	PostgreSQL 16 en contenedor Docker, con volumen persistente.	Escucha TCP en el puerto 5433.
Teléfono del usuario (opcional)	Aplicación web instalable, sin interfaz gráfica; motor de voz del sistema operativo.	HTTP y WebSocket hacia el nodo servidor por red local.
Servicio de inferencia local (opcional)	Ollama con los modelos de texto y de visión descargados.	HTTP en el puerto 11434, en la misma máquina que el servidor.

[DIAGRAMA 38] Modelo de despliegue

Qué debe mostrar: Los cinco nodos como nodos UML de despliegue, con los artefactos desplegados en cada uno y los canales de comunicación etiquetados con su protocolo y puerto. Marcar como opcionales, con notas, el teléfono y el servicio de inferencia local. Indicar que el servidor escucha en 0.0.0.0 para permitir el acceso del teléfono desde la red local.

Cómo construirlo: Diagrama de despliegue UML: cubos para los nodos, rectángulos para los artefactos y líneas de asociación con estereotipos «HTTP», «WebSocket» y «TCP».

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 38: Modelo de despliegue".

3.1.2 Diseño lógico (arquitectura en capas)

El diseño lógico organiza el sistema en cinco capas con dependencia unidireccional, de la más externa a la más interna:

Capa	Contenido	Depende de
Presentación	Vistas, componentes del lienzo, barra de herramientas, sistema de temas.	Aplicación
Aplicación (cliente)	Estado del editor, traducción de	Dominio, Infraestructura del

	acciones a operaciones, aplicación de operaciones remotas.	cliente
Dominio	Modelo UML, reglas de mapeo, análisis de normalización, generadores, adaptadores de formato. Sin dependencias externas.	Nada
Infraestructura del cliente	Cliente REST, cliente colaborativo, cola en IndexedDB, reconocimiento de voz.	Dominio
Servicio (servidor)	API REST, servidor WebSocket, capa de IA, acceso a datos.	Dominio (tipos del protocolo), Persistencia

La capa de dominio no depende de nada: ni de React, ni de la red, ni de la base de datos. Esa independencia es lo que hizo posible verificar el mapeo relacional, la normalización y el intercambio XMI ejecutándolos de forma aislada, sin levantar la interfaz.

[DIAGRAMA 39] Diagrama de capas

Qué debe mostrar: Las cinco capas apiladas con las dependencias como flechas descendentes, y la capa de dominio en el centro sin flechas salientes. Señalar con una nota que la dependencia nunca apunta hacia arriba.

Cómo construirlo: Diagrama de paquetes en disposición de capas, o diagrama de bloques apilados.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 39: Diagrama de capas".

3.2 Diseño de casos de uso

3.2.1 Diagramas de secuencia

Se especifican los diagramas de secuencia de los casos de uso que definen la arquitectura. Para los casos de uso de edición simple, el flujo es una instancia del diagrama de CU05.

CU01 – Crear pizarra colaborativa

[DIAGRAMA 40] Diagrama de secuencia: CU01 – Crear pizarra colaborativa

Qué debe mostrar: Usuario → Barra → Cliente REST → POST /api/boards → Servidor → transacción en PostgreSQL (INSERT en diagrams e INSERT en boards) → respuesta con la pizarra creada → Cliente colaborativo → conexión WebSocket y mensaje join → Servidor → snapshot vacío.

Cómo construirlo: Mostrar la transacción como un fragmento crítico: si falla el segundo INSERT, se revierte el primero.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 40: Diagrama de secuencia: CU01 – Crear pizarra colaborativa".

CU03 – Unirse a una pizarra existente

[DIAGRAMA 41] Diagrama de secuencia: CU03 – Unirse a una pizarra existente

Qué debe mostrar: Participante → Cliente colaborativo → lectura de la caché en IndexedDB → render inmediato → apertura del WebSocket → mensaje join con lastSeq → Servidor → decisión entre enviar solo las operaciones faltantes o el documento completo → difusión de presencia al resto.

Cómo construirlo: El punto importante del diagrama es la bifurcación en el servidor: usar un fragmento "alt" con las dos condiciones (atraso corto frente a atraso largo o primera conexión).

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 41: Diagrama de secuencia: CU03 – Unirse a una pizarra existente".

CU05 – Editar una clase (camino feliz)

[DIAGRAMA 42] Diagrama de secuencia: CU05 – Editar una clase (camino feliz)

Qué debe mostrar: Usuario → Nodo de clase → Estado del editor → aplicación local inmediata → construcción de la operación con su identificador → Cola offline: persistir → Cliente colaborativo: enviar → Servidor: aplicar, asignar seq, persistir operación y documento en una transacción → ack al emisor → difusión a los demás participantes → Cola offline: desencolar.

Cómo construirlo: Es el diagrama de secuencia central del sistema. Debe quedar explícito que la persistencia en la cola ocurre ANTES del envío, y que la aplicación local ocurre antes de la confirmación del servidor.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 42: Diagrama de secuencia: CU05 – Editar una clase (camino feliz)".

CU05b – Editar una clase (operación rechazada)

[DIAGRAMA 43] Diagrama de secuencia: CU05b – Editar una clase (operación rechazada)

Qué debe mostrar: Igual que el anterior hasta el servidor; el servidor determina que la operación no es aplicable —por ejemplo la clase fue eliminada— y responde con la operación en la lista de rechazadas; el cliente detecta el rechazo, desencola y solicita un resync; el servidor envía el documento completo y el cliente lo aplica.

Cómo construirlo: Este diagrama documenta la recuperación ante divergencia. Dibujarlo por separado del camino feliz para que ambos queden legibles.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 43: Diagrama de secuencia: CU05b – Editar una clase (operación rechazada)".

CU07 – Crear relación muchos a muchos

[DIAGRAMA 44] Diagrama de secuencia: CU07 – Crear relación muchos a muchos

Qué debe mostrar: Usuario → Barra: activar modo → selección de la primera clase → selección de la segunda → Estado del editor: crear la clase asociativa y las dos relaciones → envío de las tres

operaciones en un único lote, con las clases antes que las aristas → Servidor: aplicar en orden y difundir.

Cómo construirlo: Mostrar que el lote respeta el orden: el servidor rechaza una arista cuyos extremos aún no existen en el documento, por lo que las clases deben ir primero.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 44: Diagrama de secuencia: CU07 – Crear relación muchos a muchos".

CU10 – Instrucción por voz desde el teléfono

[DIAGRAMA 45] Diagrama de secuencia: CU10 – Instrucción por voz desde el teléfono

Qué debe mostrar: Usuario → PWA: tocar el micrófono → Motor de voz del dispositivo: transcribir → PWA: mostrar la transcripción → Servidor /api/ai/uml-actions → Proveedor de IA → acciones en JSON → PWA: validar y traducir a operaciones → Cola offline: persistir → envío → Servidor: aplicar y difundir → PWA: sintetizar la respuesta hablada.

Cómo construirlo: Incluir el camino sin conexión: la operación se persiste y la respuesta hablada informa que quedó en cola.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 45: Diagrama de secuencia: CU10 – Instrucción por voz desde el teléfono".

CU11 – Importar diagrama desde imagen

[DIAGRAMA 46] Diagrama de secuencia: CU11 – Importar diagrama desde imagen

Qué debe mostrar: Usuario → Barra: seleccionar imagen → Servidor POST /api/uploads: archivar la imagen → Servidor POST /api/ai/image-to-uml → Proveedor de IA con visión → clases y relaciones → Cliente: normalizar tipos y multiplicidades, resolver clases asociativas, acomodar en el lienzo → operaciones → Servidor: aplicar y difundir.

Cómo construirlo: Mostrar que el archivado de la imagen es un paso cuyo fallo no interrumpe la importación.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 46: Diagrama de secuencia: CU11 – Importar diagrama desde imagen".

CU15 – Reconexión y reconciliación

[DIAGRAMA 47] Diagrama de secuencia: CU15 – Reconexión y reconciliación

Qué debe mostrar: Cliente: detección de pérdida de red → operaciones locales que se acumulan en la cola → recuperación de la red → reenvío de la cola completa → Servidor: descarte de las operaciones ya registradas por su identificador, aceptación de las nuevas → ack → Cliente: solicitud de las operaciones faltantes → aplicación → cola vacía.

Cómo construirlo: Es el diagrama que demuestra la idempotencia. Incluir un bucle que represente varias operaciones encoladas y mostrar el descarte de duplicados en el servidor.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 47: Diagrama de secuencia: CU15 –

Reconexión y reconciliación".

CU18 – Mapeo objeto-relacional

[DIAGRAMA 48] Diagrama de secuencia: CU18 – Mapeo objeto-relacional

Qué debe mostrar: Usuario → Vista de diseño de datos → Constructor del modelo conceptual → Mapeador: aplicar R1 y R2 sobre cada clase, luego R8 sobre las herencias, luego R9 sobre las asociativas, luego R3 a R7 sobre las relaciones restantes → Vista: presentar esquema y trazabilidad.

Cómo construirlo: El orden de aplicación de las reglas importa y debe verse en el diagrama: primero las tablas, después la herencia, después las asociativas, y al final las asociaciones simples.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 48: Diagrama de secuencia: CU18 – Mapeo objeto-relacional".

CU21 – Generar el backend

[DIAGRAMA 49] Diagrama de secuencia: CU21 – Generar el backend

Qué debe mostrar: Usuario → Barra → Diálogo de opciones de base de datos → Generador: por cada clase, emitir entidad, repositorio, servicio y controlador; emitir pom.xml, properties y clase principal; en la opción PostgreSQL, emitir los archivos de Docker → Generador de Postman: derivar la colección → Empaquetador: construir el ZIP → Navegador: descargar.

Cómo construirlo: Usar un bucle sobre las clases del diagrama para no repetir el mismo intercambio.

Herramienta: Diagrama de secuencia UML (draw.io, PlantUML o Architech)

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 49: Diagrama de secuencia: CU21 – Generar el backend".

3.3 Diseño de datos

Este apartado documenta el diseño de datos del propio sistema, es decir, la base sobre la cual la herramienta persiste las pizarras y los diagramas. No debe confundirse con el esquema que la herramienta genera para el usuario, que es el resultado del módulo documentado en CU17 a CU20 y que depende del diagrama que se modele en cada caso.

3.3.1 Modelo conceptual

El dominio persistente está formado por cuatro entidades. La decisión de diseño más relevante es que el diagrama se almacena de dos formas complementarias: como documento materializado en una columna JSONB, para poder leerlo de una sola vez; y como secuencia de operaciones en una tabla aparte, que constituye la historia del documento y permite poner al día a un cliente atrasado sin transferir todo el diagrama.

Entidad	Descripción	Tipo
Diagram	Documento colaborativo: el contenido del diagrama y su versión, expresada como número	Fuerte

	de secuencia.	
Board	Vista de trabajo sobre un diagrama, con su nombre. Es la unidad que el usuario ve y comparte.	Débil: no existe sin su diagrama.
Operation	Una operación aceptada sobre un diagrama, con su lugar en el orden total, su autor y su contenido.	Débil: no existe sin su diagrama.
Upload	Metadatos de una imagen archivada. El binario se guarda en el sistema de archivos.	Fuerte

[DIAGRAMA 50] Modelo conceptual de la base de datos del sistema

Qué debe mostrar: Las cuatro entidades con sus atributos y las relaciones con sus cardinalidades: Diagram 1—1 Board (composición: la pizarra no existe sin su diagrama); Diagram 1—* Operation (composición); Upload sin relaciones. Marcar las claves primarias y señalar Board y Operation como entidades débiles.

Cómo construirlo: Diagrama entidad-relación, o diagrama de clases con estereotipos de persistencia. Las dos composiciones justifican el ON DELETE CASCADE del esquema.

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 50: Modelo conceptual de la base de datos del sistema".

3.3.2 Mapeo objeto-relacional

Aplicando las reglas descritas en la fundamentación teórica, el modelo conceptual produce cuatro tablas. Se indica para cada una su clave primaria y sus claves foráneas.

diagrams

PK					
id	name	doc	seq	created_at	updated_at

Regla R1 sobre la entidad Diagram. La columna doc es el documento materializado y seq su versión.

boards

PK		FK	
id	name	diagram_id	created_at

Regla R1 sobre Board más regla R6: la foránea hacia diagrams es obligatoria y borra en cascada, por ser una composición.

diagram_ops

PK	FK						
op_id	diagram_id	seq	client_id	actor_name	kind	payload	created_at

Regla R1 sobre Operation más regla R6. La clave primaria es el identificador generado por el cliente, no un serial: esa es la decisión que hace idempotente la sincronización. Restricción de unicidad adicional sobre (diagram_id, seq) para garantizar el orden total.

uploads

PK				
id	filename	mime_type	size_bytes	created_at

Regla R1 sobre Upload. Sin relaciones.

3.3.3 Diseño físico y tabla de volúmenes

Detalle de cada tabla con el tipo de dato, la obligatoriedad y el tipo de clave.

diagrams

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de llave
id	UUID	Identificador del diagrama	16 bytes	NO	Primaria
name	TEXT	Nombre del diagrama	Variable	NO	
doc	JSONB	Documento materializado: clases, relaciones y marcas de borrado	Variable	NO	
seq	BIGINT	Versión del documento: número de la última operación aplicada	8 bytes	NO	
created_at	TIMESTAMPTZ	Fecha de creación	8 bytes	NO	
updated_at	TIMESTAMPTZ	Fecha de la última modificación	8 bytes	NO	

boards

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de llave
id	TEXT	Identificador de la pizarra	Variable	NO	Primaria
name	TEXT	Nombre visible de la pizarra	Variable	NO	
diagram_id	UUID	Diagrama sobre	16 bytes	NO	Foránea →

		el que trabaja la pizarra			diagrams.id
created_at	TIMESTAMPTZ	Fecha de creación	8 bytes	NO	

diagram_ops

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de llave
op_id	UUID	Identificador de la operación, generado por el cliente	16 bytes	NO	Primaria
diagram_id	UUID	Diagrama al que pertenece	16 bytes	NO	Foránea → diagrams.id
seq	BIGINT	Posición en el orden total del diagrama	8 bytes	NO	Única junto con diagram_id
client_id	TEXT	Cliente que originó la operación	Variable	NO	
actor_name	TEXT	Nombre visible del autor, para la auditoría	Variable	Sí	
kind	TEXT	Tipo de operación: alta, modificación, movimiento, baja o reemplazo	Variable	NO	
payload	JSONB	Contenido de la operación	Variable	NO	
created_at	TIMESTAMPTZ	Momento en que el servidor la aceptó	8 bytes	NO	

uploads

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de llave
id	UUID	Identificador del archivo	16 bytes	NO	Primaria
filename	TEXT	Nombre del archivo en disco	Variable	NO	
mime_type	TEXT	Tipo de contenido	Variable	NO	
size_bytes	BIGINT	Tamaño en bytes	8 bytes	NO	
created_at	TIMESTAMPTZ	Fecha de subida	8 bytes	NO	

Índices definidos

Índice	Tabla y columnas	Por qué
diagram_ops_by_diagram_seq	diagram_ops (diagram_id, seq)	Es la consulta caliente del

		sistema: recuperar las operaciones de un diagrama posteriores a un número de secuencia, que es lo que se ejecuta cada vez que un cliente reconecta.
boards_by_diagram	boards (diagram_id)	Resolución de la pizarra a partir del diagrama.
UNIQUE (diagram_id, seq)	diagram_ops	Garantiza a nivel de base de datos que no existan dos operaciones con el mismo número de secuencia en un mismo diagrama: es la que protege el orden total incluso ante un error de programación en el servidor.
PRIMARY KEY (op_id)	diagram_ops	Garantiza la idempotencia: un reenvío de la misma operación es rechazado por la base de datos, no solo por la lógica de la aplicación.

Las dos últimas restricciones son deliberadamente redundantes con la lógica del servidor. La razón es que las garantías de consistencia deben estar en la base de datos: si el código de aplicación tuviera un defecto, la base rechaza la operación en lugar de corromper el orden total.

[DIAGRAMA 51] Modelo lógico relacional de la base de datos del sistema

Qué debe mostrar: Las cuatro tablas con todas sus columnas, tipos, claves primarias y foráneas, las reglas de borrado y los índices. Debe verse la relación uno a uno entre boards y diagrams, y la uno a muchos entre diagrams y diagram_ops, ambas con CASCADE.

Cómo construirlo: Diagrama relacional. Se puede obtener automáticamente desde el gestor de base de datos, o dibujarlo en la propia herramienta del proyecto y exportar el modelo lógico desde la vista de diseño de datos.

Herramienta: DBeaver, pgAdmin, o la vista de diseño de datos del propio proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 51: Modelo lógico relacional de la base de datos del sistema".

3.3.4 Ejemplo del esquema generado para el usuario

Para ilustrar el resultado del módulo de diseño de datos se incluye el DDL producido a partir de un diagrama de ejemplo con las cuatro clases Mascota, Dueño, Consulta y Veterinario, con una composición, una asociación y una agregación. Este script fue ejecutado contra una base PostgreSQL real como parte de las pruebas.

- Esquema relacional generado a partir del diagrama de clases
- Reglas de mapeo aplicadas: R1..R9 (transformación objeto-relacional, Rumbaugh)

```

-- Origen: Clase Dueno
CREATE TABLE dueno (
  -- Clave primaria subrogada (regla R1)
  id BIGSERIAL NOT NULL,
  -- Atributo private nombre: String (regla R2)
  nombre VARCHAR(255),
  telefono VARCHAR(255),
  PRIMARY KEY (id)
);

-- Origen: Clase Mascota
CREATE TABLE mascota (
  id BIGSERIAL NOT NULL,
  nombre VARCHAR(255),
  raza VARCHAR(255),
  fecha_nacimiento DATE,
  -- Relación composicion 1:N con Dueno (regla R4 + R6)
  dueno_id BIGINT NOT NULL,
  PRIMARY KEY (id),
  CONSTRAINT fk_mascota_dueno_id FOREIGN KEY (dueno_id)
    REFERENCES dueno(id) ON DELETE CASCADE
);

CREATE INDEX idx_mascota_dueno_id ON mascota (dueno_id);

```

NOTA PARA COMPLETAR: Reemplazar este ejemplo por el DDL del diagrama que se vaya a defender, si el docente pide un caso concreto. Se obtiene con el botón "Descargar .sql" de la vista de diseño de datos.

4. Flujo de trabajo: Implementación

4.1 Implementación de la arquitectura del sistema

El sistema se implementó como un repositorio con tres proyectos independientes: la infraestructura de datos, el servidor y el cliente.

proyecto-software-1/ ├─ docker-compose.yml ├─ README.md / SETUP.md ├─ server/ │ ├── sql/schema.sql │ ├── test-collab.mjs │ └── src/ │ ├── config.ts │ ├── db.ts │ ├── types.ts │ ├── collab.ts │ └── rooms.ts └─ transaccional ├── ws.ts ├── index.ts ├── ai/ └── routes/ └─ uml-board/ ├── public/ └── src/ ├── styles/theme.css ├── lib/ ├── store/ ├── components/ ├── services/ ├── utils/ ├── pages/ └── routes/	PostgreSQL de la aplicación documentación de puesta en marcha backend propio (Node + TypeScript) esquema: diagrams, boards, diagram_ops, uploads suite de pruebas de colaboración configuración por variables de entorno pool de conexiones y migración tipos del protocolo colaborativo aplicación de operaciones sobre el documento orden total por diagrama y persistencia sesiones, presencia y puesta al día arranque del servidor HTTP y WebSocket proveedores de inferencia y prompts REST: pizarras, diagramas, bitácora, IA, imágenes cliente (React + TypeScript + Vite) manifiesto e ícono de la PWA, service worker tokens de diseño; se redefinen para el tema oscuro infraestructura del cliente estado del editor barra, menús, nodos y aristas del diagrama IA y aplicación de acciones dominio: diseño de datos, generadores, adaptadores pizarra, diseño de datos, guía, voz, diagnóstico enrutamiento
--	---

[DIAGRAMA 52] Diagrama de componentes de la implementación

Qué debe mostrar: Los componentes físicos con sus interfaces: el cliente expone las cinco vistas y consume dos interfaces del servidor (REST y WebSocket); el servidor expone esas dos interfaces y consume la interfaz de PostgreSQL y las de los dos proveedores de IA. Mostrar los artefactos que produce el sistema como componentes de salida: el ZIP del backend Spring Boot, la colección de Postman, el DDL y el archivo XML.

Cómo construirlo: Diagrama de componentes UML con interfaces dibujadas como círculos (notación de bola y zócalo).

Herramienta: Architech, o la propia herramienta del proyecto

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 52: Diagrama de componentes de la implementación".

4.2 Implementación de la arquitectura del subsistema

Subsistema de colaboración

Es el núcleo del sistema y donde se concentran las decisiones no evidentes. Su implementación se apoya en cuatro mecanismos:

- **Operación granular:** cada acción del usuario produce una operación con un tipo —alta, modificación, movimiento, baja o reemplazo—, un identificador único generado en el cliente, el autor y el contenido mínimo del cambio. Las modificaciones se expresan como fusión parcial por campo, no como reemplazo del elemento, de modo que dos personas que editan campos distintos de la misma clase no se pisan.
- **Serialización por diagrama:** en el servidor, todas las operaciones de un mismo diagrama se aplican de una en una a través de una cadena de promesas. Sin esa serialización, dos operaciones concurrentes podrían obtener el mismo número de secuencia.
- **Transacción única:** la operación se registra en la bitácora y el documento se actualiza en la misma transacción. Si la transacción falla, el estado en memoria del diagrama se descarta para forzar su recarga desde la base de datos, evitando que la memoria quede adelantada respecto de lo persistido.
- **Marcas de borrado:** el documento conserva una lista acotada de identificadores eliminados. Una operación tardía sobre un elemento borrado se descarta en lugar de resucitarlo.

Subsistema de trabajo sin conexión

La cola de operaciones y la caché del documento viven en IndexedDB. El orden de las operaciones en el flujo es deliberado: primero se aplica el cambio en la vista, después se persiste la operación, y solo entonces se intenta enviarla. Si el envío falla, la operación ya es durable. Al reconectar se reenvía la cola completa y el servidor descarta lo que ya tenía.

Un detalle que se corrigió durante la implementación: la pérdida de red no siempre cierra el socket, porque el navegador puede mantenerlo abierto y encolar los envíos. En ese caso el intento de reapertura no hace nada y el indicador quedaba mostrando "sin conexión" de forma permanente. La solución fue que, al recibir el evento de recuperación de red, si el socket sigue abierto se restaure el estado y se reenvíe la cola explícitamente.

Subsistema de inteligencia artificial

Los prompts y las credenciales viven en el servidor. La interfaz del proveedor recibe un mensaje de sistema, un mensaje de usuario y opcionalmente una imagen, y devuelve texto junto con el nombre del proveedor y del modelo que respondieron, para que la interfaz pueda mostrarlo. Ambos proveedores se configuran para devolver JSON: el modelo local mediante el modo de formato JSON de Ollama y el de la nube mediante el formato de respuesta estructurada. Aun así, el cliente aplica un parseo tolerante, porque los modelos a veces envuelven el JSON en marcas de código.

Configuración y seguridad

Ninguna credencial está incrustada en el código fuente. El servidor lee su configuración de variables de entorno y el cliente solo conoce la dirección del servidor. El repositorio incluye

plantillas de configuración de ejemplo, y los archivos de configuración reales están excluidos del control de versiones.

NOTA PARA COMPLETAR: Durante la migración se detectó que el sistema base heredado contenía una clave de API de OpenAI y dos claves de proyecto de un servicio de base de datos escritas en el código fuente y publicadas en un repositorio público. Se eliminaron del código. Si esas credenciales siguen activas, deben revocarse. Conviene mencionar esta corrección en la defensa: es un hallazgo de seguridad real y su corrección es parte del mantenimiento preventivo documentado.

5. Flujo de trabajo: Prueba

5.1 Planificar el plan de pruebas

Elemento del plan	Descripción aplicada al proyecto
Alcance de la prueba	Se prueba: la corrección del motor de colaboración ante concurrencia, desconexión y reenvío; la corrección del esquema y del DDL generados; la fidelidad del intercambio XMI en ida y vuelta; y el funcionamiento de la interfaz completa en navegadores reales. No se prueba de forma automatizada la calidad de la interpretación de la IA, por ser no determinística; se verifica manualmente.
Estrategia	Caja blanca para las funciones de dominio (mapeo, normalización, adaptadores), que son puras y verificables de forma aislada. Caja negra de extremo a extremo para la colaboración y la interfaz, con clientes reales contra el servidor real y la base de datos real. Pruebas de integración para los límites entre subsistemas.
Criterios de aceptación	El type-check y el linter deben pasar sin errores en ambos proyectos. El DDL generado debe ejecutarse sin errores sobre PostgreSQL y producir el comportamiento semántico esperado en las reglas de borrado. Tres sesiones concurrentes deben converger al mismo estado. El reenvío de la cola no debe producir duplicados. El intercambio XMI no debe perder elementos.
Recursos	PostgreSQL 16, Node.js 22, navegador Chromium controlado por Playwright, Postman para el backend generado, y los scripts de prueba incluidos en el repositorio.
Riesgos de la prueba	La ausencia de un proveedor de IA disponible impide probar los casos de uso que dependen de inferencia; por eso la estrategia híbrida se considera parte del diseño para la testabilidad y no solo una comodidad.

5.2 Diseñar las pruebas

Atributo	Detalle del diseño
Casos de uso relacionados	CU05, CU06, CU14 y CU15: edición concurrente, presencia y trabajo sin conexión.
Descripción	Verificar que tres clientes que editan simultáneamente el mismo diagrama converjan al mismo estado, que dos ediciones sobre campos distintos de la misma clase sobrevivan ambas, que el trabajo hecho sin conexión se sincronice al

	reconectar, que el reenvío de la cola no duplique, y que una operación tardía sobre un elemento borrado sea rechazada.
Condiciones de entrada	Servidor y base de datos en ejecución; una pizarra vacía; tres clientes con identidades distintas conectados al mismo diagrama.
Resultado esperado	Los tres clientes ven las mismas clases; el renombre y el agregado de atributo sobre la misma clase coexisten; tras reconectar, la cola queda vacía y el resto ve los cambios; el segundo envío del mismo lote no incrementa el número de clases; la clase eliminada no reaparece.
Procedimiento	Ejecutar el script <code>server/test-collab.mjs</code> con el servidor en ejecución. El script abre tres conexiones WebSocket, simula la desconexión cerrando una de ellas, y verifica el estado final consultando la API.

5.3 Implementar las pruebas

Las pruebas se implementaron en tres niveles, todos ejecutables desde el repositorio:

Nivel	Implementación	Qué verifica
Dominio (caja blanca)	Ejecución directa de las funciones de mapeo, normalización y adaptadores sobre un diagrama de prueba, comparando el resultado con el esperado.	Reglas de mapeo, orden de creación de tablas en el DDL, fidelidad del XMI en ida y vuelta.
Colaboración (caja negra)	<code>server/test-collab.mjs</code> : tres clientes WebSocket contra el servidor real y PostgreSQL real.	Convergencia, fusión por campo, cola offline, idempotencia, marcas de borrado y bitácora de auditoría.
Interfaz de extremo a extremo	Navegadores Chromium reales controlados por Playwright, con contextos aislados para simular usuarios distintos y con simulación de pérdida de red.	Presencia y propagación a través de la interfaz, indicador de cola, reconexión, menús, control de zoom, persistencia del tema y las cinco rutas.
Verificación del DDL	Ejecución del script generado contra una base PostgreSQL creada al efecto, más inserciones y borrados de comprobación.	Que el DDL sea ejecutable y que las reglas de borrado tengan el comportamiento semántico correcto.

5.4 Realizar pruebas de integración

ID	Interfaz involucrada	Descripción	Resultado
INT-01	Cliente ↔ Servidor	Incorporación a un	Éxito: los tres clientes

	(WebSocket)	diagrama, recepción del documento y difusión de operaciones a los demás participantes.	convergen al mismo estado y la presencia se refleja en los tres.
INT-02	Servidor ↔ PostgreSQL	Registro de la operación en la bitácora y actualización del documento en una sola transacción.	Éxito: el estado persistido coincide con el de los clientes; la bitácora registra 22 operaciones con su autor en la corrida de referencia.
INT-03	Cliente ↔ IndexedDB	Persistencia de la operación antes del envío y desencolado tras la confirmación.	Éxito: el indicador muestra la operación en cola sin conexión y la cola queda vacía tras reconectar.
INT-04	Servidor ↔ Proveedor de IA	Envío del prompt con el contexto del diagrama y obtención de las acciones en JSON.	Éxito con proveedor disponible. Sin proveedor, el sistema informa la causa y no modifica el diagrama, que es el comportamiento especificado.
INT-05	Módulo de diseño de datos ↔ PostgreSQL	Ejecución del DDL generado sobre una base real.	Éxito: 17 tablas y 10 claves foráneas creadas sin errores a partir del diagrama de prueba.
INT-06	Adaptador XMI ↔ modelo UML	Exportación e importación del mismo diagrama, comparando el resultado.	Éxito con una pérdida conocida y documentada: los atributos de tipo Date regresan como String, porque XMI 2.5 no define un primitivo Date.
INT-07	PWA móvil ↔ Servidor	Operación por voz desde el teléfono sobre la pizarra compartida.	Éxito: la instrucción se aplica y el resto de los participantes la recibe.

5.5 Realizar pruebas de sistema

ID	Funcionalidad evaluada	Descripción	Resultado
SYS-01	Tres sesiones colaborativas simultáneas	Tres participantes crean una clase cada uno al mismo tiempo.	Pasado: los tres ven las tres clases. Con el diseño anterior, basado en guardar el documento completo, este escenario perdía

			trabajo.
SYS-02	Edición concurrente del mismo elemento	Un participante renombra una clase mientras otro le agrega un atributo.	Pasado: ambos cambios sobreviven, porque la modificación se fusiona por campo.
SYS-03	Trabajo sin conexión	Un participante pierde la red, sigue editando y reconecta.	Pasado: el indicador muestra la cola, los cambios locales se conservan, y al reconectar se propagan al resto.
SYS-04	Reenvío duplicado de la cola	Se reenvía el mismo lote de operaciones, simulando la pérdida de la confirmación.	Pasado: el número de clases no cambia; el servidor descarta los duplicados por identificador.
SYS-05	Borrado contra edición tardía	Un participante elimina una clase y otro envía después una operación sobre ella.	Pasado: la clase no reaparece; el cliente detecta el rechazo y resincroniza.
SYS-06	Generación completa del backend	Desde un diagrama, obtener el proyecto Spring Boot.	Pasado: se descarga el ZIP con el proyecto Maven completo.
SYS-07	Diseño de datos de extremo a extremo	Del diagrama al DDL ejecutado, verificando las reglas de borrado.	Pasado: la composición propaga el borrado y la agregación anula la referencia, como especifican las reglas R6 y R7.
SYS-08	Aplicación móvil por voz	Instalación de la PWA y operación con un solo control.	Pasado: la pantalla presenta un único control y responde por voz.
SYS-09	Tema claro y oscuro	Alternancia, persistencia entre recargas y seguimiento de la preferencia del sistema.	Pasado en los tres casos.
SYS-10	Degradación de la capa de IA	Comportamiento sin ningún proveedor disponible.	Pasado: el sistema informa la ausencia en la vista de diagnóstico y en la barra, y las funciones que no dependen de IA siguen operativas.

NOTA PARA COMPLETAR: Antes de la entrega conviene volver a ejecutar las pruebas y actualizar esta tabla con los resultados de esa corrida, para que la documentación refleje el estado exacto de la versión que se defiende. Los comandos están en el apartado de pruebas del README.

5.6 Evaluar la prueba

Criterio	Estado	Observación
Consistencia ante concurrencia	Conforme	Verificado con tres sesiones simultáneas y con edición del mismo elemento. Es el atributo de calidad priorizado y el defecto principal del sistema base heredado.
Funcionamiento ante desconexión	Conforme	Cola durable, reenvío idempotente y reconciliación verificados en navegadores reales.
Corrección del diseño de datos	Conforme	DDL ejecutado sobre PostgreSQL con verificación del comportamiento de las reglas de borrado.
Generación de código	Conforme	El proyecto generado se empaqueta completo y respeta la estructura en capas.
Interfaz asistida por IA	Conforme con dependencia externa	Funciona con proveedor local o de nube; sin ninguno, la función se deshabilita e informa. La calidad de la interpretación depende del modelo y no es determinística.
Interoperabilidad CASE	Parcialmente conforme	XMI 2.5 verificado en ida y vuelta. El adaptador nativo de Architech no está implementado.
Calidad del código	Conforme	Type-check y linter sin errores en el cliente y en el servidor.
Seguridad de credenciales	Conforme	Ninguna credencial en el código fuente; configuración por variables de entorno excluidas del control de versiones.
Autenticación y control de accesos	No conforme	No implementado. Se declara como limitación conocida.

Parte III – Guías de usuario

El enunciado admite que la guía de usuario sea un agente inteligente. Se implementaron las dos formas de manera complementaria: una guía escrita, que es la que sigue, y un agente dentro de la aplicación que responde preguntas en lenguaje natural.

1. La guía como agente inteligente

La vista de guía de la aplicación responde preguntas escritas con las palabras del usuario. Está construida en dos capas, y ese orden es una decisión de diseño:

4. Base de conocimiento local: un conjunto de preguntas y respuestas escritas, con búsqueda por coincidencia de términos clave, que se ejecuta en el dispositivo. Funciona sin servidor y sin inteligencia artificial.
5. Capa de inferencia: si la búsqueda local no encuentra respuesta, la pregunta se envía al proveedor de IA junto con esa misma base de conocimiento como contexto obligatorio, con la instrucción explícita de responder solo con lo que la documentación cubre y de declarar cuando no la cubre.

La razón de ese orden es la disponibilidad: una guía de usuario que depende de un servicio externo deja de existir cuando ese servicio no está, que es justamente el momento en que el usuario más necesita ayuda. Además, entregar la base de conocimiento como contexto evita que el modelo describa funciones que la herramienta no tiene. La interfaz indica siempre si la respuesta proviene de la guía escrita o de la IA.

2. Puesta en marcha

Requisitos: Node.js 18 o superior, y Docker para la base de datos. Opcionalmente Ollama para la inteligencia artificial local.

Paso 1 – Base de datos

```
docker compose up -d
docker compose ps      # debe indicar "healthy"
```

Paso 2 – Servidor

```
cd server
Copy-Item .env.example .env
npm install
npm run dev
```

El servidor aplica el esquema por sí solo la primera vez y crea una pizarra inicial. Verificación: la dirección `http://localhost:4000/health` debe responder que la base está en pie.

Paso 3 – Cliente

```
cd uml-board
Copy-Item .env.example .env
npm install
npm run dev
```

La aplicación queda disponible en <http://localhost:5173>.

NOTA PARA COMPLETAR: Los comandos están escritos para PowerShell de Windows. En PowerShell el separador de comandos es el punto y coma, no la doble ampersand.

Paso 4 – Inteligencia artificial

Para la IA local, instalar Ollama y descargar los modelos:

```
ollama pull qwen2.5:7b-instruct      # texto, ~4,7 GB
ollama pull llama3.2-vision:11b     # visión, ~7,9 GB
```

Para la IA en la nube, colocar la clave propia en la variable `OPENAI_API_KEY` del archivo `server/.env`. La estrategia recomendada es la híbrida, que usa la local y recurre a la nube solo si aquella falla.

Paso 5 – Aplicación móvil

6. Obtener la dirección IP del equipo en la red local.
7. Colocar esa dirección en `VITE_API_URL` dentro de `uml-board/.env`.
8. Arrancar el cliente con la opción que lo expone a la red local.
9. Desde el teléfono, en la misma red, abrir la dirección terminada en `/voz`.
10. Instalarla con la opción "Agregar a la pantalla de inicio" del navegador.

El micrófono requiere HTTPS o localhost en la mayoría de los navegadores; si el teléfono no permite el micrófono por HTTP, el origen puede habilitarse manualmente en la configuración del navegador.

3. Guía de uso por tarea

3.1 Modelar un diagrama de clases

11. Presionar el botón "+ Clase" en la barra superior: la clase aparece en el lienzo.
12. Escribir el nombre de la clase directamente sobre ella.
13. Agregar atributos indicando nombre, tipo y visibilidad.
14. Para relacionar dos clases, iniciar el modo relación desde la clase de origen, elegir el tipo y las multiplicidades, y hacer clic en la clase de destino.
15. Para una relación muchos a muchos, usar la opción correspondiente del menú Modelar: crea la clase asociativa y sus dos relaciones automáticamente.

3.2 Modelar hablando o escribiendo

En el menú Modelar, la opción del asistente abre una caja donde se puede escribir o dictar la instrucción. Ejemplos que el sistema interpreta:

- "Creá una clase Mascota con nombre texto, raza texto y fecha de nacimiento fecha."
- "Agregá el atributo peso decimal a Mascota."
- "Mascota se relaciona con Dueño, uno a muchos."
- "Consulta hereda de Atención."

- “Relación muchos a muchos entre Médico y Paciente.”

3.3 Importar un diagrama de una foto

En Intercambio, la opción de importar desde una foto permite subir una imagen del diagrama. Para obtener buenos resultados la foto debe estar de frente, enfocada y con los nombres legibles. El resultado es un diagrama editable, no definitivo: conviene revisarlo y corregirlo.

3.4 Trabajar en equipo

16. En el menú de la pizarra, elegir "Copiar enlace para invitar".
17. Compartir ese enlace con quienes vayan a participar.
18. Cada participante puede cambiar su nombre visible desde el indicador de presencia.
19. La barra muestra cuántas personas están conectadas y, al abrirla, quién está editando qué.

3.5 Obtener el diseño de datos y el backend

20. En el menú Generar, la opción de diseño de datos abre la vista con las cinco etapas: modelo conceptual, mapeo, esquema, normalización y DDL.
21. Revisar las observaciones de normalización: cada una viene con su justificación y su corrección propuesta.
22. Descargar el informe completo en formato Markdown, o el script SQL del dialecto elegido.
23. En el menú Generar, la opción de backend Spring Boot pide elegir la base de datos y descarga el proyecto completo en un ZIP.
24. Descomprimir y ejecutar el proyecto con Maven; importar en Postman la colección incluida para probar los endpoints.

3.6 Cambiar el tema

El botón de sol o luna al final de la barra alterna entre claro y oscuro con un clic. Con clic derecho sobre ese mismo botón se puede elegir "Seguir al sistema", para que acompañe la configuración del sistema operativo. La elección se recuerda.

4. Resolución de problemas

Síntoma	Causa probable y solución
La barra muestra “Sin conexión” y “IA no disponible” al mismo tiempo.	El servidor no está en ejecución: las dos indicaciones dependen de él. Verificar que la terminal de la carpeta server siga activa y que la base esté arriba.
La aplicación abre pero el diagrama está vacío.	El cliente está mostrando la caché local sin haber podido sincronizar. Revisar la vista de diagnóstico.
El asistente responde con un error de IA.	No hay ningún proveedor disponible. La vista de diagnóstico indica el estado de cada uno por separado.

La importación desde foto no reconoce nada.	El modelo de visión no está disponible, o la foto no es legible. Probar con una imagen más nítida y de frente.
El teléfono no se conecta.	El cortafuegos del equipo bloquea los puertos en la red privada, o la dirección configurada en el cliente no es la IP del equipo.
El micrófono no funciona en el teléfono.	El navegador exige HTTPS o localhost para el micrófono. Habilitar el origen manualmente en la configuración del navegador.

[DIAGRAMA 53] Capturas de pantalla de la guía de usuario

Qué debe mostrar: Insertar capturas de: la pizarra con un diagrama en tema claro y en tema oscuro; un menú de la barra desplegado; la vista de diseño de datos en la pestaña de normalización; la vista de DDL; la guía de usuario con una pregunta respondida; y la aplicación móvil con el control de voz.

Cómo construirlo: Tomar las capturas con la aplicación en ejecución y un diagrama de ejemplo cargado. Conviene usar el mismo diagrama en todas, para que la guía se lea como un recorrido continuo.

Herramienta: Captura de pantalla del sistema operativo

Reemplazar este bloque por la imagen del diagrama y su epígrafe "Figura 53: Capturas de pantalla de la guía de usuario".

NOTA PARA COMPLETAR: Grabar además el video de la guía de usuario y enlazarlo en el QR de la portada. Un recorrido de cinco a siete minutos que muestre: modelado manual, una instrucción por voz, la importación de una foto, tres sesiones colaborativas simultáneas, la desconexión y reconexión con la cola, la vista de diseño de datos y la generación del backend probado en Postman.

BIBLIOGRAFÍA

- Jacobson, I., Booch, G. y Rumbaugh, J. (1999). El Proceso Unificado de Desarrollo de Software. Addison-Wesley.
- Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F. y Lorensen, W. (1991). Object-Oriented Modeling and Design. Prentice Hall.
- Rumbaugh, J., Jacobson, I. y Booch, G. (2005). The Unified Modeling Language Reference Manual (2.^a ed.). Addison-Wesley.
- Object Management Group (2015). OMG Unified Modeling Language (OMG UML), Version 2.5. Recuperado de <https://www.omg.org/spec/UML/2.5/>
- Object Management Group (2015). XML Metadata Interchange (XMI) Specification, Version 2.5.1. Recuperado de <https://www.omg.org/spec/XMI/>
- Pressman, R. S. y Maxim, B. R. (2020). Ingeniería del Software: un enfoque práctico (9.^a ed.). McGraw-Hill.
- Sommerville, I. (2016). Software Engineering (10.^a ed.). Pearson.
- Bass, L., Clements, P. y Kazman, R. (2021). Software Architecture in Practice (4.^a ed.). Addison-Wesley.
- Date, C. J. (2003). An Introduction to Database Systems (8.^a ed.). Addison-Wesley.
- Elmasri, R. y Navathe, S. B. (2016). Fundamentals of Database Systems (7.^a ed.). Pearson.
- Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387.
- Szyferski, C. (2002). Component Software: Beyond Object-Oriented Programming (2.^a ed.). Addison-Wesley.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
- Shapiro, M., Preguiça, N., Baquero, C. y Zawirski, M. (2011). Conflict-Free Replicated Data Types. INRIA, Rapport de recherche RR-7687.
- ISO/IEC (2011). ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.
- ISO (2019). ISO 9241-210: Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems.
- VMware (2024). Spring Boot Reference Documentation, Version 3.3. Recuperado de <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- VMware (2024). Spring Data JPA Reference Documentation. Recuperado de <https://docs.spring.io/spring-data/jpa/reference/>

World Wide Web Consortium (2020). Web Speech API Specification. Recuperado de <https://wicg.github.io/speech-api/>

World Wide Web Consortium (2018). Indexed Database API 2.0. Recuperado de <https://www.w3.org/TR/IndexedDB-2/>

Internet Engineering Task Force (2011). RFC 6455: The WebSocket Protocol.

PostgreSQL Global Development Group (2024). PostgreSQL 16 Documentation. Recuperado de <https://www.postgresql.org/docs/16/>

Ollama (2024). Ollama Documentation. Recuperado de <https://ollama.com/>

NOTA PARA COMPLETAR: Revisar y completar las fechas de consulta de las fuentes en línea según el formato de citación que exija el docente (APA, IEEE u otro).

ANEXOS

Anexo A - Limitaciones conocidas

Este anexo declara de forma explícita lo que el sistema no hace. Se incluye por dos razones: porque la documentación debe describir el software que efectivamente existe, y porque durante la defensa de autoría cualquier diferencia entre lo documentado y lo implementado es un problema mayor que la limitación misma.

Limitación	Situación actual	Qué haría falta
Autenticación y control de accesos	No implementado. Cualquiera que tenga el enlace de una pizarra puede editarla, y todos los participantes tienen las mismas capacidades. La distinción entre anfitrión y participante es funcional, no de permisos.	Un módulo de identidad con sesión, y un modelo de permisos por pizarra. La bitácora de operaciones ya registra el autor, de modo que la trazabilidad está preparada.
Bloqueo de elementos	El sistema evita la pérdida de trabajo por fusión de operaciones, y muestra qué clase está tocando cada participante, pero no impide que dos personas editen el mismo elemento a la vez.	Un bloqueo cooperativo sobre el elemento seleccionado, con expiración por tiempo. La infraestructura de presencia ya transporta la selección.
Formato nativo de Architech	No implementado. La arquitectura de adaptadores está definida y el adaptador figura deshabilitado en la interfaz. La exportación e importación funcionan en XMI 2.5, que es el formato estándar del OMG, y en el formato JSON propio.	Un archivo de ejemplo exportado desde Architech para determinar su formato. Con eso, implementar el adaptador y habilitarlo; no requiere modificar el resto del sistema.
Tipo Date en el intercambio XMI	Los atributos de tipo Date se exportan y regresan como String.	Es una limitación del estándar: XMI 2.5 no define un primitivo Date. Podría resolverse con una anotación propia, al costo de reducir la portabilidad.
Operaciones de las clases	El editor modela atributos pero no operaciones (métodos).	No son necesarias para derivar el esquema de datos ni las entidades del backend, que es el alcance definido. Incorporarlas requeriría extender el modelo y el generador.
Otros diagramas UML	Solo se modela el diagrama de clases.	Es el alcance declarado: el diagrama de clases es el que permite derivar datos y código. Los demás diagramas de esta documentación se construyen

		con herramientas externas.
Pruebas automatizadas de la interpretación de la IA	Se verifica manualmente.	La salida de un modelo generativo no es determinística, por lo que una aserción de igualdad no es aplicable. Sería necesario un conjunto de pruebas con tolerancia, evaluando si el resultado satisface la intención y no si coincide exactamente.
Deshacer y rehacer	No implementado en la interfaz.	La bitácora de operaciones contiene la información necesaria; faltaría definir la operación inversa de cada tipo y la semántica del deshacer en un contexto colaborativo, que no es trivial.

Anexo B – Estándar de codificación

B.1 Introducción

El proyecto adopta un estándar de codificación verificable de forma automática, con la premisa de que un estándar que depende de la disciplina individual no se cumple. Las reglas están configuradas en herramientas que fallan la compilación o el análisis cuando se incumplen.

B.2 Estándares internacionales aplicados

Característica (ISO/IEC 25010)	Implementación técnica	Evidencia
Mantenibilidad – Modularidad	Separación en capas con dependencia unidireccional y una capa de dominio sin dependencias externas.	server/src/, uml-board/src/utils/
Mantenibilidad – Reusabilidad	Componentes con interfaz definida: adaptadores de formato, proveedores de IA, generadores.	utils/architech.ts, server/src/ai/provider.ts
Mantenibilidad – Analizabilidad	TypeScript con modo estricto en cliente y servidor; sin tipos implícitos.	tsconfig.app.json, server/tsconfig.json
Mantenibilidad – Modificabilidad	Sistema de tokens de diseño: los colores se definen una sola vez y ningún componente escribe valores literales.	uml-board/src/styles/theme.css
Fiabilidad – Tolerancia a fallos	Reconexión con espera creciente, cola durable, y degradación del proveedor de IA.	lib/collabClient.ts, lib/offlineQueue.ts, ai/provider.ts

Fiabilidad – Recuperabilidad	Resincronización ante divergencia y transacciones en la persistencia.	server/src/rooms.ts
Seguridad – Confidencialidad	Credenciales fuera del código fuente, en variables de entorno excluidas del control de versiones.	server/.env.example, .gitignore
Usabilidad – Operabilidad	Barra agrupada por intención, tema claro y oscuro, indicadores de estado de conexión y de cola.	components/Toolbar.tsx, lib/theme.tsx
Usabilidad – Accesibilidad	Etiquetas accesibles en los controles de ícono, foco visible, y respeto de la preferencia de movimiento reducido.	styles/theme.css, components/

B.3 Convenciones de código

- **Nomenclatura:** camelCase para variables y funciones, PascalCase para tipos y componentes, UPPER_SNAKE_CASE para constantes, snake_case para columnas de base de datos.
- **Archivos:** PascalCase para componentes de interfaz, camelCase para módulos de lógica.
- **Comentarios:** se comenta el por qué de una decisión, no el qué hace la línea. Los comentarios que explican una decisión no evidente son obligatorios; los que narran el código son ruido.
- **Tipado:** prohibido el tipo implícito; el uso de un tipo dinámico debe estar justificado con un comentario y una anotación explícita.
- **Manejo de errores:** ningún bloque de captura vacío sin un comentario que explique por qué el error es ignorable.
- **Idioma:** identificadores del dominio en español, coherentes con el vocabulario de la materia; términos técnicos consolidados en inglés.

B.4 Herramientas de verificación

Herramienta	Qué verifica	Comando
TypeScript (modo estricto)	Tipos, variables y parámetros sin usar, y exhaustividad.	npm run type-check
ESLint	Reglas de React y de TypeScript, tipos dinámicos sin justificar, importaciones sin usar.	npm run lint
Prettier	Formato uniforme: comillas, longitud de línea, sangrías.	npm run format:check
Husky y lint-staged	Ejecuta el análisis y el formateo sobre los archivos modificados antes de cada commit.	automático
Compilación	Que el proyecto completo compile y se empaquete.	npm run build

El estado de verificación al momento de esta entrega es: type-check y ESLint sin errores en el cliente y en el servidor, y compilación exitosa de ambos proyectos.

B.5 Estructura de carpetas y separación de responsabilidades

La estructura refleja la arquitectura, no el tipo de archivo. Cada carpeta agrupa lo que cambia por la misma razón, y esa correspondencia entre estructura y arquitectura es lo que permite ubicar rápidamente dónde se implementa cada requerimiento. El detalle está en el apartado 4.1 de la Parte II.

Anexo C – Enlaces del proyecto

Recurso	QR	Enlace
Repositorio de GitHub	(insertar)	(completar)
Software publicado	(insertar)	(completar)
Video de la guía de usuario	(insertar)	(completar)
Documentación publicada	(insertar)	(completar)

NOTA PARA COMPLETAR: Completar esta tabla y la portada con los enlaces definitivos. Recordar que la documentación debe estar publicada el día de la entrega como máximo hasta las 8:00, como condición para asistir a la presentación presencial.

Anexo D – Índice de diagramas por construir

El documento especifica 53 diagramas. Cada bloque marcado en el cuerpo del documento indica qué debe mostrar el diagrama, cómo construirlo y con qué herramienta. Este anexo los lista en orden para poder llevar el control de lo que falta.

NOTA PARA COMPLETAR: Al terminar cada diagrama, reemplazar su bloque de instrucción en el cuerpo del documento por la imagen, con el epígrafe "Figura N: título". Luego eliminar este anexo y actualizar el índice con F9.

N.º	Diagrama	Herramienta sugerida	Hecho
01	Diagrama de arquitectura general del sistema	Architech, o la propia herramienta del proyecto	☐
02	Diagrama de clases del propio sistema (modelo de dominio)	Architech, o la propia herramienta del proyecto	☐
03	Ciclo de vida PUDS del proyecto	draw.io, Lucidchart o PowerPoint	☐
04	Diagrama de secuencia de la capa de IA en modo híbrido	Architech, o la propia herramienta del proyecto	☐
05	Diagrama de actores y su relación con el sistema	Architech, o la propia herramienta del proyecto	☐
06	Caso de uso individual CU01 – Crear pizarra colaborativa	Architech, o la propia herramienta del proyecto	☐
07	Caso de uso individual CU02 – Compartir enlace de invitación	Architech, o la propia herramienta del proyecto	☐
08	Caso de uso individual CU03 – Unirse a una pizarra existente	Architech, o la propia herramienta del proyecto	☐
09	Caso de uso individual CU04 – Administrar pizarras	Architech, o la propia herramienta del proyecto	☐
10	Caso de uso individual CU05 – Crear y editar clases con sus atributos	Architech, o la propia herramienta del proyecto	☐
11	Caso de uso individual CU06 – Crear relaciones entre clases	Architech, o la propia herramienta del proyecto	☐
12	Caso de uso individual CU07 – Crear relación muchos a muchos con clase asociativa	Architech, o la propia herramienta del proyecto	☐
13	Caso de uso individual CU14 – Visualizar los participantes conectados y su actividad	Architech, o la propia herramienta del proyecto	☐
14	Caso de uso individual	Architech, o la propia	☐

	CU15 – Trabajar sin conexión y sincronizar al reconectar	herramienta del proyecto	
15	Caso de uso individual CU16 – Consultar la bitácora de operaciones del diagrama	Architech, o la propia herramienta del proyecto	☐
16	Caso de uso individual CU08 – Editar el diagrama mediante instrucción escrita	Architech, o la propia herramienta del proyecto	☐
17	Caso de uso individual CU09 – Editar el diagrama mediante instrucción por voz	Architech, o la propia herramienta del proyecto	☐
18	Caso de uso individual CU10 – Operar el sistema por voz desde el teléfono	Architech, o la propia herramienta del proyecto	☐
19	Caso de uso individual CU11 – Importar un diagrama desde una fotografía o boceto	Architech, o la propia herramienta del proyecto	☐
20	Caso de uso individual CU24 – Consultar la guía de usuario	Architech, o la propia herramienta del proyecto	☐
21	Caso de uso individual CU25 – Consultar el diagnóstico del sistema y el estado de la IA	Architech, o la propia herramienta del proyecto	☐
22	Caso de uso individual CU12 – Importar un diagrama desde archivo	Architech, o la propia herramienta del proyecto	☐
23	Caso de uso individual CU13 – Exportar el diagrama a otra herramienta CASE	Architech, o la propia herramienta del proyecto	☐
24	Caso de uso individual CU17 – Generar el modelo conceptual de datos	Architech, o la propia herramienta del proyecto	☐
25	Caso de uso individual CU18 – Generar el mapeo objeto-relacional y el esquema lógico	Architech, o la propia herramienta del proyecto	☐
26	Caso de uso individual CU19 – Analizar la normalización del esquema	Architech, o la propia herramienta del proyecto	☐
27	Caso de uso individual CU20 – Generar el DDL de la base de datos	Architech, o la propia herramienta del proyecto	☐

28	Caso de uso individual CU21 – Generar el backend en Spring Boot	Architech, o la propia herramienta del proyecto	☐
29	Caso de uso individual CU22 – Generar la colección de pruebas para Postman	Architech, o la propia herramienta del proyecto	☐
30	Caso de uso individual CU23 – Generar el frontend de prueba	Architech, o la propia herramienta del proyecto	☐
31	Diagrama general de casos de uso	Architech, o la propia herramienta del proyecto	☐
32	Casos de uso del Ciclo 1	Architech, o la propia herramienta del proyecto	☐
33	Casos de uso del Ciclo 2	Architech, o la propia herramienta del proyecto	☐
34	Casos de uso del Ciclo 3	Architech, o la propia herramienta del proyecto	☐
35	Diagrama de paquetes	Architech, o la propia herramienta del proyecto	☐
36	Diagrama de clases de análisis de CU15 (trabajo sin conexión)	Architech, o la propia herramienta del proyecto	☐
37	Diagrama de clases de análisis de CU08 (instrucción por IA)	Architech, o la propia herramienta del proyecto	☐
38	Modelo de despliegue	Architech, o la propia herramienta del proyecto	☐
39	Diagrama de capas	Architech, o la propia herramienta del proyecto	☐
40	Diagrama de secuencia: CU01 – Crear pizarra colaborativa	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
41	Diagrama de secuencia: CU03 – Unirse a una pizarra existente	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
42	Diagrama de secuencia: CU05 – Editar una clase (camino feliz)	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
43	Diagrama de secuencia: CU05b – Editar una clase (operación rechazada)	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
44	Diagrama de secuencia: CU07 – Crear relación muchos a muchos	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
45	Diagrama de secuencia: CU10 – Instrucción por voz desde el teléfono	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
46	Diagrama de secuencia: CU11 – Importar diagrama desde imagen	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐

47	Diagrama de secuencia: CU15 – Reconexión y reconciliación	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
48	Diagrama de secuencia: CU18 – Mapeo objeto-relacional	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
49	Diagrama de secuencia: CU21 – Generar el backend	Diagrama de secuencia UML (draw.io, PlantUML o Architech)	☐
50	Modelo conceptual de la base de datos del sistema	Architech, o la propia herramienta del proyecto	☐
51	Modelo lógico relacional de la base de datos del sistema	DBeaver, pgAdmin, o la vista de diseño de datos del propio proyecto	☐
52	Diagrama de componentes de la implementación	Architech, o la propia herramienta del proyecto	☐
53	Capturas de pantalla de la guía de usuario	Captura de pantalla del sistema operativo	☐